

יחידה 6: רשתות נשנות (RNN)

עיבוד מקדים של נתוני טקסט

ביחידה זו נלמד על ארכיטקטורות רשת מיוחדות עבור נתונים סדרתיים, כגון מדידות של אוסף משתנים בפרקי זמן קבועים (למשל מדידה יומית של טמפרטורה ולחות), או משפטים בשפה טבעית, שבהם לסדר הופעת המילים השפעה על משמעות המשפט. כדוגמה חשבו על ההבדל בין שני המשפטים שלהלן, המשתמשים באותן המילים, רק בסדר שונה:

"טוב מאוד, לא רע"

"רע מאוד, לא טוב"

אם ברצוננו לאמן אלגוריתם לזהות את הרגש הבא לידי ביטוי בכל אחד מהמשפטים הנ"ל (חיובי או שלילי), הוא יהיה חייב ללמוד שסדר המילים במשפט משפיע על הרגש המובע. רשת בעלת קישוריות מלאה המקבלת כקלט את המשפט כולו תוכל ללמוד את החשיבות של הסדר, אך יידרשו לכך דוגמאות רבות ותהליך אימון סבוך. על כן, כמו ביחידת הלימוד הקודמת שבה תיכננו מראש רשתות המיועדות למשימות עיבוד תמונה, ביחידה זו נתכנן רשתות המביאות בחשבון באופן מובנה את ממד הסדר של הנתונים. רשתות אלו מכונות רשתות נשנות (Recurrent Neural Networks – RNN). הדוגמה אשר תלווה אותנו היא אוסף משפטים בשפה האנגלית המביעים רגש שלילי או חיובי, ובפרק הנוכחי נלמד על עבודת ההכנה הדרושה לפני הזנתם כקלט לרשת נוירונים.

אוסף הנתונים

מאגר המשאבים GLUE מורכב מתשעה אוספי נתונים שונים בתחום עיבוד השפה הטבעית המשמשים לבדיקת ביצועיהם של אלגוריתמים לעיבוד שפה טבעית (Natural Language Processing – NLP) במגוון משימות שונות. אחת המשימות היא זיהוי הרגש (שלילי או חיובי) המובע במשפט נתון, ואוסף הנתונים הנבחר למשימה זו הוא SST2: אוסף של כ-70,000 משפטים המתויגים לשתי מחלקות: 0 – רגש שלילי, 1 – רגש חיובי.

אוסף נתונים זה (ורבים אחרים), ניתן לטעון בעזרת הספרייה Datasets, אך ראשית יש להתקין אותה בסביבת העבודה, שכן היא לא מותקנת כברירת מחדל בהפצות הסטנדרטיות של פייתון. לשם כך, הריצו את הפקודה הזו.

```
pip install datasets
```

לאחר ההתקנה ניתן לייבא את הספרייה, ובשורת קוד אחת לטעון את הנתונים הרצויים.

```
import datasets as ds
dataset = ds.load_dataset("glue", "sst2")
```

מתוך אוסף הנתונים ניקח את 67,000 המשפטים המשמשים לאימון, ונטען לזיכרון את המשפטים וסיווגם כרשימות סטנדרטיות של פייתון.

```
sentence_list = dataset["train"]["sentence"]
labels_list = dataset["train"]["label"]
```

כעת נדפיס משפט אחד וסיווגו, כדי לוודא שטעינת הנתונים בוצעה כהלכה.

```
print(sentence_list[1], labels_list[1], sep="\n")
```

פלט:

```
contains no wit , only labored gags
0
```

עיבוד מקדים

לפני הזנת הנתונים לאלגוריתם הלמידה, עלינו לערוך בהם עיבוד מקדים. בעוד יכולנו להזין נתונים ויזואליים, כגון תמונות, ישירות לרשתות קונבולוציה, אין זה המצב עבור רשתות נשנות: עלינו להמיר את המשפטים לטנזורים מספריים בצורה שיטתית, כאשר השלב הראשון בתהליך הוא טוקניזציה (tokenization) של המשפט, כלומר חלוקתו לרכיבים קטנים. הגישה הבסיסית ביותר למשימה זו היא חלוקת המשפט למילים לפי סימן הרווח המופיע ביניהן, כדלהלן

```
sentence = sentence_list[1]
print(sentence, type(sentence))
print(sentence.split())
```

פלט:

```
contains no wit , only labored gags <class 'str'>
['contains', 'no', 'wit', ',', 'only', 'labored', 'gags']
```

שימוש במתודה `split` על משתנה מטיפוס `str` יניב רשימה של משתנים מאותו טיפוס – אלו הם הטוקנים (tokens) של המשפט המקורי.

השלב הבא בתהליך העיבוד הוא יצירת אוצר המילים: איסוף כל הטוקנים באוסף הנתונים, ומיפויים למספרים סידוריים באופן חד־חד־ערכי. לצורך זה נשתמש בספרייה `torchtext`.

```
from torchtext.vocab import build_vocab_from_iterator
tokenizer = lambda x: x.split()
tokenized = list(map(tokenizer, sentence_list))

vocab = build_vocab_from_iterator(tokenized)
print(vocab(sentence.split()))
print(vocab("one two three".split()))

[2923, 60, 329, 1, 88, 1992, 548]
[28, 128, 356]
```

פלט:

ראו כיצד בעזרת הפונקציה map ביצענו טוקניזציה לכל המשפטים בשורת קוד אחת. המשפטים הועברו לבנאי אוצר המילים המובנה, אשר החזיר אובייקט אוצר מילים המקודד טוקנים למספרים סידוריים. כך ניתן לראות למשל שהמספר 28 מייצג את המילה "one".

מכיוון שכל מספר שייך למילה אחת בלבד, ניתן לפרש קידוד נתון חזרה לטוקנים המתאימים. בדוגמה שלהלן אנחנו מפרשים את סדרת עשרת המספרים הראשונים חזרה לטוקנים, אלו הם הטוקנים הנפוצים ביותר באוסף הנתונים.

```
print(vocab.get_itos()[0:10]) # integer to string

['the', ',', 'a', 'and', 'of', '.', 'to', "'s", 'is', 'that']
```

פלט:

אוצר המילים נבנה על בסיס סט נתוני האימון, אך לאחר אימון האלגוריתם נרצה להפעיל אותו על קלט כלשהו, לא דווקא כזה המורכב מאוצר המילים הקיים. במצב הנוכחי, אם ננסה לבצע טוקניזציה והמרה למספרים סידוריים למשפט שמכיל מילים חדשות נקבל שגיאה.

```
vocab("hello world".split())

RuntimeError: Token hello not found and default index is not set
```

פלט:

על כן נוסיף טוקן מיוחד עבור מילים מחוץ לאוצר המילים, ובאותה הזדמנות נמחק ממנו מילים נדירות, שכן אוצר מילים גדול מאוד יכביד על תהליך הלמידה.

```
vocab = build_vocab_from_iterator(tokenized,
                                specials=["<UNK>"], min_freq=5)
vocab.set_default_index(0)
vocab("hello world".split())

[0, 152]
```

פלט:

כעת אנו רואים כי אף שהמילה "hello" לא נמצאת באוצר המילים, לא התקבלה שגיאה – מילה זו מופתה לטוקן של המילים הלא ידועות.

לבסוף, כל שנותר לעשות הוא לחלק את כל המשפטים באוסף הנתונים לטוקנים ולהמירם למספרים הסידוריים שלהם באוצר המילים.

```
stoi = lambda x: torch.tensor(vocab(x)) # string to integer
integer_tokens = list(map(stoi, tokenized))
print(integer_tokens[1])
```

פלט:

```
tensor([2924, 61, 330, 2, 89, 1993, 549])
```

במשתנה `integer_tokens` התקבלה רשימה פייתונית של טנזורים המכילים מספרים טבעיים: המספרים הסידוריים באוצר המילים של טוקני המשפטים המקוריים.

רשתות נוירונים פועלות מטבען על מספרים ממשיים ולא טבעיים. על כן, השלב האחרון בעיבוד הנתונים יהיה להמיר את המספרים הסידוריים לייצוג ממשי: לכל מילה נתאים וקטור ממשי במרחב רב-ממדי, זהו שלב שיכון המילה (word embedding).

```
embedding_layer = nn.Embedding(len(vocab), 3)
integer = stoi(["word"])
print(integer)
print(embedding_layer(integer))
```

פלט:

```
tensor([1234])
tensor([[ 0.4685,  1.1884, -0.3000]],
grad_fn=<EmbeddingBackward0>)
```

בקוד זה הגדרנו שכבה המבצעת שיכון מילים למרחב תלת-ממדי, המרנו את המילה "word" למספר הסידורי שלה במילון: 1234 וחישבנו את השיכון שלו. כאשר במשפט נתון תופיע המילה "word", טנזור תלת-ממדי זה הוא שיוזן לרשת הנוירונים במקומה.

שימו לב כי מערכת הגזירה האוטומטית עוקבת אחרי השיכונים – אלו הם פרמטרים אשר יילמדו יחד עם אימון הרשת, מתוך ציפייה שהאלגוריתם ילמד לייצג את המילים בצורה מועילה לניתוח הרגש של משפט נתון. בקטע הקוד שלהלן נדגים כיצד השיכונים שמורים בשכבה: כמטריצה אשר השורה ה־א'ית שלה מכילה את השיכון של המילה ה־א'ית באוצר המילים.

```
W = embedding_layer.weight
print(W.size())
print(W[integer,:])
```

פלט:

```
torch.Size([11222, 3])
tensor([[ 0.4685,  1.1884, -0.3000]], grad_fn=<IndexBackward0>)
```

למידת שיכונים מוצלחים היא משימה מאתגרת לא פחות מאימון רשת הנוירונים עצמה: בדרך כלל השיכון יהיה במרחב מממד גבוה, ובאוצר המילים קיימות עשרות אלפי מילים. על כן, רק בשכבת השיכון ייתכן שיהיו יותר פרמטרים מבכל שאר הרשת. כדי להתמודד עם אתגר זה, ניתן להשתמש בשיכונים מאומנים מראש (pretrained embeddings) על אוסף נתונים גדול. בקטע הקוד שלהלן נדגים

כיצד לטעון את הגרסה הקטנה ביותר של GloVe: שיכוני מילים אשר אומנו על כל ערכי ויקיפדיה בשנת 2014.

```
from torchtext.vocab import GloVe
glove_embedder = GloVe(name='6B', dim=50)
embedded = glove_embedder.get_vecs_by_tokens(["It's", "cool"])
print(embedded.size(), embedded.dtype, embedded.requires_grad)
```

פלט:

```
torch.Size([2, 50]) torch.float32 False
```

העברת משפט לאחר טוקניזציה ל-GloVe תניב טנזור המכיל את השיכונים של המילים במרחב 50-ממדי, שאותם נזין לרשת הנוירונים במקום פלט שכבת השיכון שהגדרנו בעבר. שימו לב שמערכת ה-autograd אינה עוקבת אחרי שיכוני המילים, שכן ערכי השיכון כבר מאומנים ואין לנו רצון שהם ישתנו בתהליך אימון הרשת אשר עתיד לבוא.

במבט עמוק יותר לתוצאה נגלה כי השיכון של המילה "It's" הוא טנזור האפס: זהו השיכון של מילים מחוץ לאוצר המילים של GloVe. בעת שימוש בשיכון מאומן מראש עלינו לתת את הדעת גם לאוצר המילים ולתהליך הטוקניזציה שבו השתמשו בעת למידת השיכון, מאחר שנרצה להתאים אליהם את תהליך עיבוד הנתונים הגולמיים שלנו ככל האפשר.

נוסף על כך, יש להביא בחשבון גם את העובדה ששיכונים אלו לא אומנו על אוסף הנתונים שאיתו אנו עובדים, וכן לא למטרת סיווג הרגש המובע במשפט, כך שיתכן שאין הם אידיאליים למטרה זו. זאת ועוד, אוסף הנתונים הגדול ששיכונים אלו אומנו עליו לא מתאים ללמידה מפקחת, שכן אין זה סביר לתייג את הרגש המובע בכל משפט בוויקיפדיה: עלות התיוג באופן אנושי תהיה יקרה להחריד. עקב כך, שיכוני GloVe נלמדו בפיקוח עצמי (Self Supervised Learning – SSL): כשלב הכנה לאלגוריתם האימון חושבו מספר המופעים המשותפים של כל זוג מילים באוסף המשפטים. מטריצת המופעים המשותפים בתורה שימשה ליצירת פונקציית המטרה של אלגוריתם הלמידה, כאשר התוצאה היא שכל הלמידה התרחשה על אוסף נתונים לא מתויג, מבלי להידרש להתערבות אנושית ביצירת אוסף הנתונים. עקב כך, מילים המופיעות לרוב יחדיו באוסף הנתונים הן בעלות שיכונים קרובים זה לזה.

שאלה לתרגול

המילים במשפט "It's cool" נפוצות בשימוש ועל כן לא סביר שהן לא נלמדו בתהליך האימון של GloVe. נסו לבצע טוקניזציה שונה למשפט, ידנית, ובדקו אם בדרך זו אפשר למצוא שיכון בעל ערך לכל טוקן. **הערה:** טעינת השיכונים של GloVe כרוכה בהורדת נתונים בנפח של כ-1GB.

סיווג נתוני טקסט

בפרק זה נאמן רשת פשוטה מאוד לזיהוי הרגש הבא לידי ביטוי במשפט מאוסף הנתונים SST2. בפרקים הבאים נשלב בתוכה את רכיבי הארכיטקטורה המתקדמים, תוך כדי פיתוחם. בדומה לבעיית סיווג הנקודות השחורות והלבנות לשתי מחלקות המוכרת לנו מיחידה 2, גם במקרה זה אנו מעוניינים לסווג לשתי מחלקות. על כן, בסופה של הרשת יהיה עלינו למקם ראש סיווג המניב פלט בעל שני איברים ומעביר אותו לפונקציית softmax. לפיכך על ראש הסיווג להיות מוגדר כדלהלן.

```
class ClassificationHead(nn.Module):
    def __init__(self, in_features):
        super().__init__()
        self.linear = nn.Linear(in_features, 2)
        self.logsoftmax = nn.LogSoftmax(dim=0)

    def forward(self, feature_extractor_output):
        class_scores = self.linear(feature_extractor_output)
        logprobs = self.logsoftmax(class_scores)
        return logprobs
```

ראו כי פלט ראש הסיווג הוא לוגריתם הסתברויות הסיווג, וזכרו כי נשתמש בו עבור פונקציית המחיר המתאימה לבעיית סיווג: האנטרופיה הצולבת.

כעת עלינו להגדיר את שאר הרשת, אשר יחלץ מאפיינים מהמשפט, ונרצה לעשות זאת בצורה הפשוטה ביותר: נזין ישירות את פלט שכבת השיכון אל ראש הסיווג. אם כן, מחלץ המאפיינים יורכב מהשיכון בלבד ובפרקים הבאים נהפוך אותו למתוחכם יותר.

```
class FeatureExtractor(nn.Module):
    def __init__(self, embed_dim):
        super().__init__()
        self.embedding = nn.Embedding(len(vocab), embed_dim)

    def forward(self, sentence_tokens):
        embedded = self.embedding(sentence_tokens)
        return embedded
```

בהינתן משפט לאחר טוקניזציה והמרת הטוקנים למספרים הסידוריים שלהם, מחלץ המאפיינים יחזיר טנזור בעל שני ממדים: ממדו הראשון יציין את מיקום הטוקן במשפט, וממדו השני את מרחב השיכון. ראו לדוגמה,

```

print(example_sentence)

preprocess = lambda x: torch.tensor(vocab(x.split()))
tokens     = preprocess(example_sentence)
print(tokens)

extractor = FeatureExtractor(2)
features  = extractor(tokens)
print(features, features.size(), sep="\n")

contains no wit , only labored gags
tensor([2924,  61, 330,  2,  89, 1993, 549])
tensor([[ 0.9569, -0.6598],
        [ 0.9742, -1.0970],
        [-0.3451,  0.7615],
        [-0.8656,  1.8823],
        [ 0.6175,  0.2272],
        [ 0.9894, -0.8952],
        [ 1.0751, -1.2522]], grad_fn=<EmbeddingBackward0>)
torch.Size([7, 2])

```

פלט:

פלט זה, המשתנה בגודלו בהתאם לאורך המשפט, עתיד להתנגש עם הקלט הצפוי לראש הסיווג: בעת
 אתחול ראש הסיווג יש להגדיר את מספר ניווני הקלט בשכבה הלינארית, בפרמטר `in_features`
 ובכך לקבוע את גודל הקלט. כדי להתגבר על קושי זה, נסכום את כל שיכוני הטוקנים במשפט נתון
 ליצירת טנזור אשר ממדו קבוע, והוא כממד מרחב השיכון. השינוי הדרוש במתודת `forward` של
 המחלק מופיע בקטע הקוד שלהלן.

```

def forward(self, sentence_tokens):
    embedded      = self.embedding(sentence_tokens)
    feature_extractor_output = embedded.sum(dim=0)
    return feature_extractor_output

```

כעת, כאשר ממד הפלט של מחלק המאפיינים ידוע מראש, ניתן לחברו לראש הסיווג ולקבל את הרשת
 השלמה.

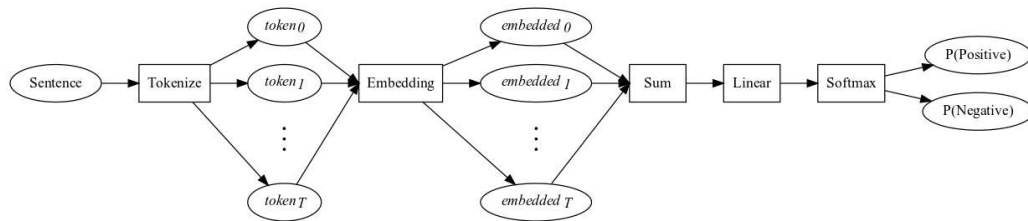
```

class EmbedSumClassify(nn.Module):
    def __init__(self, embed_dim):
        super().__init__()
        self.extractor = FeatureExtractor(embed_dim)
        self.classifier = ClassificationHead(embed_dim)

    def forward(self, sentence_tokens):
        extracted_features = self.extractor(sentence_tokens)
        logprobs          = self.classifier(extracted_features)
        return logprobs

```

תהליך עיבוד הנתונים והזנתם לרשת לצורך קבלת הסתברויות הסיווג מאויר להלן.



רשת זו אינה ערוכה לקבל batch של משפטים באורכים שונים, אך תוך כדי הזנת המשפטים זה אחר זה, נוכל לאמנה כרגיל.

רשת בסיסית כנ"ל מסוגלת ללמוד לסווג את המשפטים באופן סביר, אולם היא סובלת ממגבלה קריטית: פעולת הסכום מבטלת את משמעות סדר הופעת המילים במשפט. ראו בדוגמה שלהלן כיצד שני משפטים הנבדלים זה מזה רק בסדר הופעת המילים מסווגים באופן זהה.

```
example_sentences = ["very good , not bad",
                    "very bad , not good"]
with torch.no_grad():
    for sent in example_sentences:
        print(preprocess(sent))
        print(torch.exp(model(preprocess(sent))))
```

פלט:

```
tensor([77, 46, 2, 33, 74])
tensor([1.0000e+00, 4.7996e-07])
tensor([77, 74, 2, 33, 46])
tensor([1.0000e+00, 4.7996e-07])
```

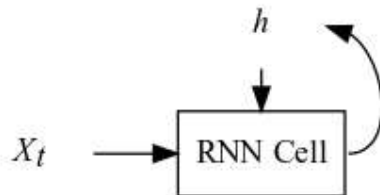
אין זה מפתיע, שכן הטוקנים של שני המשפטים זהים, ועל כן הקלט לראש הסיווג עבור שניהם זהה, ויישאר כך גם לאחר האימון.

שאלות לתרגול

- הגדירו אובייקט EmbedSumClassify, הזינו את משפטי סט האימון לרשת והעבירו את הפלט המתקבל לפונקציית המחיר של האנטרופיה הצולבת. אמנו את הרשת לסיווג הרגש המובע במשפטים, תוך כדי הזנת המשפטים זה אחר זה. בדקו את ביצועי הרשת המאומנת עבור ממדי שיכון שונים.
- החליפו את שכבת השיכון ופעולת הסכום בשכבת nn.EmbeddingBag, המבצעת שתי פעולות אלו זו אחרי זו ביעילות רבה יותר ואמנו את המודל מחדש. השוו את זמני הריצה. שימו לב שהקלט הצפוי של שכבה זו הוא batch של משפטים, ולכן בעת הזנת משפט יחיד עליכם להוסיף ממד מנוון בעזרת המתודה unsqueeze, ולהוריד אותו לאחר מכן.
- החליפו את שכבת השיכון בשיכוני GloVe והשוו את התוצאה לרשת עם שיכוני נלמדים. השוו גם את זמני הריצה של תהליך האימון והסבירו את הפער שמצאתם.

רשתות נשנות

בפרק זה נכיר את רכיב הרשת אשר יאפשר לרשת הנורונית להביא בחשבון את סדר הופעת המילים במשפט, זהו תא הרשת הנשנית (RNN cell) הקרוי לעתים גם תא אלמן (Elman cell), על שם החוקר הראשון שפרסם רשתות המבוססות עליו. בהמשך הדיון נניח כי משפט הקלט עבר טוקניזציה ושיכון, וכן ש- X_t הוא השיכון של הטוקן ה- t במשפט. הטוקנים מוזנים לפי הסדר אל תא הרשת הנשנית, שבו ישנו מצב חבוי (hidden state) h אשר מתעדכן עם הזנת כל טוקן. מלבד הטוקן, גם המצב החבוי הקודם משפיע על החישוב המתבצע בתא, וכך טוקנים המופיעים בתחילת המשפט משפיעים על המשך החישוב דרך השארת חותמם על המצב החבוי. להלן מצויר היזון חוזר זה באופן סכמתי,



ביתר פירוט, בעת הזנת X_t לתא, המצב החבוי הבא מחושב ונשמר לפי הנוסחה

$$h_{t+1} = \tanh(W_{input}X_t + b_{input} + W_{hidden}h_t + b_{hidden})$$

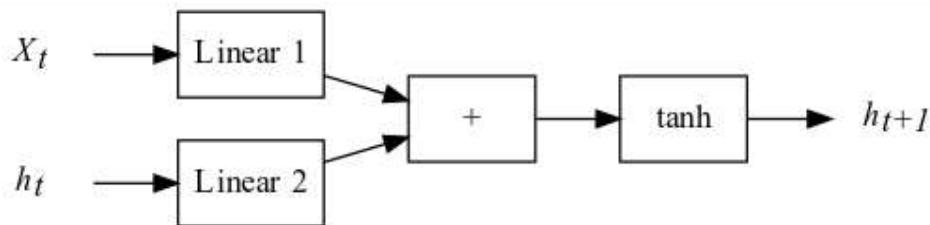
כאשר $W_{input}, b_{input}, W_{hidden}, b_{hidden}$ הם הפרמטרים הנלמדים של התא, וכן

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

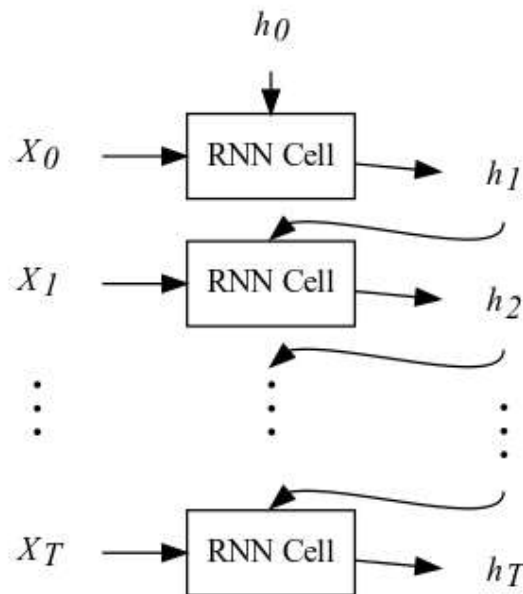
היא פונקציית הטנגנס ההיפרבולי: פונקציית אקטיבציה דמוית סיגמואיד אשר מחזירה ערכים בטווח $(-1, 1)$. ניתן לפרק חישוב זה לשלבים:

1. הזנת טוקן הקלט (לאחר שיכון), X_t , לאגרגציה לינארית בעלת הפרמטרים W_{input}, b_{input} .
2. הזנת המצב החבוי הקודם לאגרגציה לינארית **אחרת** בעלת הפרמטרים W_{hidden}, b_{hidden} .
3. סכימת פלט שכבות האגרגציה.
4. הפעלת האקטיבציה על הסכום.
5. שמירת המצב החבוי החדש.

ובאור,



בעת מעבר על דגימה אחת, משפט באורך T טוקנים, החישוב הנ"ל מתבצע בטור T פעמים, ואם נרצה לאייר את זרימת המידע ברשת בפירוט, עלינו להביא זאת בחשבון. ראו באיור שלהלן כיצד אנו פורסים (unfold) את איור התא המקורי לאורך ממד הזמן, ומביאים לידי ביטוי מפורש את החישוב החוזר על עצמו. ודאו כי אתם מבינים שמדובר בתא נשנה יחיד, בעל אותם פרמטרים.



כעת נממש תא אלמון בקוד כמודול של PyTorch, ולאחר מכן נראה כיצד לשלבו ברשת נוירונים לחיזוי הרגש המובע במשפט.

```
class MyRNNCell(nn.Module):
    def __init__(self, embed_dim, hidden_dim):
        super().__init__()
        self.hidden_state = torch.zeros(hidden_dim)
        self.hidden_linear = nn.Linear(in_features = hidden_dim,
                                         out_features = hidden_dim)
        self.input_linear = nn.Linear(in_features = embed_dim,
                                         out_features = hidden_dim)
        self.activation = nn.Tanh()
    def forward(self, one_embedded_token):
        Z1 = self.input_linear(one_embedded_token)
        Z2 = self.hidden_linear(self.hidden_state)
        Y = Z1+Z2
        new_state = self.activation(Y)
        self.hidden_state = new_state
        #return
```

שימו לב לכך שהמצב החבוי מוגדר מראש להיות בעל `hidden_dim` מאפיינים, שהוא מאותחל באפסים, וכן שבעת מעבר קדימה יחיד מוזן לתא טוקן משוכן בגודל `embed_dim`. בשונה משכבות שראינו עד כה – לא מוחזר כל ערך בסוף החישוב, אלא המצב החבוי עצמו מעודכן ונשמר בתא. ראו גם כיצד מוגדרות השכבות הלינאריות, כך שפעולת הסכום תתבצע בין טנזורים מאותו ממד, וכן שממד המצב החבוי החדש יישאר זהה לממד המצב החבוי הקודם.

בבואנו לממש את הרשת הנשנית לסיווג המשפטים, נעשה שינויים אחדים בתוך הרשת שכתבנו בפרק הקודם: נוותר על פעולת סכום השיכונים אשר ביטלה את חשיבות סדר הופעתם במשפט, ובמקומה נזין את שיכונים הטוקנים לפי הסדר לתוך התא. על המצב החבוי האחרון, לאחר הזנת כל הטוקנים, נחשוב בתור פלט מחלף המאפיינים, ונזין אותו כרגיל לשכבה לינארית ולפונקציית `softmax`.

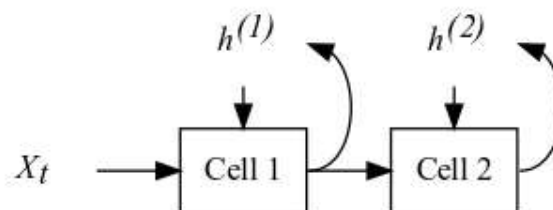
```
class RNNClassifier(nn.Module):
    def __init__(self, embed_dim, hidden_dim):
        super().__init__()
        self.hidden_dim = hidden_dim
        self.embedding = nn.Embedding(len(vocab), embed_dim)
        self.rnn = MyRNNCell(embed_dim, hidden_dim)
        self.linear = nn.Linear(hidden_dim, 2)
        self.logsoftmax = nn.LogSoftmax(dim=0)

    def forward(self, sentence_tokens):
        self.rnn.hidden_state = torch.zeros(self.hidden_dim)
        for one_token in sentence_tokens:
            one_embedded_token = self.embedding(one_token)
            self.rnn(one_embedded_token)

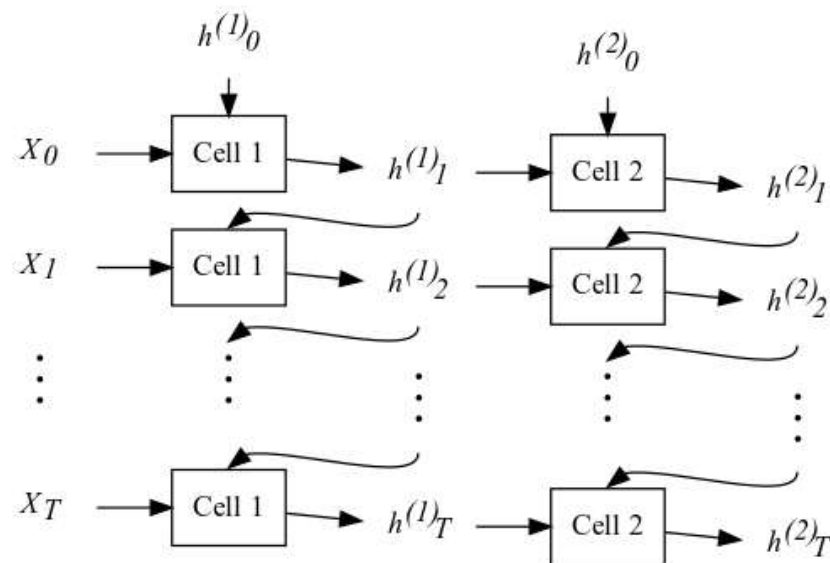
        feature_extractor_output = self.rnn.hidden_state
        class_scores = self.linear(feature_extractor_output)
        logprobs = self.logsoftmax(class_scores)
        return logprobs
```

ראו כיצד הצורך בחישוב המצב החבוי החדש על סמך המצב החבוי הקודם מוביל לשימוש בלולאה בעת במעבר קדימה ברשת: לפני חישוב המצב החבוי הסופי יש לחשב את כל הקודמים לו בסדרה.

כמו ברשתות בעלות קישוריות מלאה ורשתות קונבולוציה, ריבוי שכבות ועיבוד המידע לעומק השכבות מועיל גם עבור רשתות נשנות, אך במימוש הנ"ל אנו משתמשים רק בתא יחיד. כדי להרחיב את הרשת הנשנית לרשת עמוקה, עלינו לחבר תאי אלמן זה לזה, כך שהמצב החבוי של התא הקודם יועבר כקלט לתא הבא, במקום הטוקן המשוכן. ובאיור סכמתי,



כמו כן, נאייר את החישוב המתבצע בתאים לאחר פריסה לאורך ממד הזמן,



כעת פלט מחלץ המאפיינים יהיה פלט התא השני, לאחר הזנת כל הטוקנים, המסומן ב- $h_T^{(2)}$ באיור הנ"ל. נממש רשת בעלת שתי שכבות נשנות באמצעות שינוי קוד המודול הקודם במעט, כאשר השינויים מסומנים ב-#.

```
class DeepRNNClassifier(nn.Module):
    def __init__(self, embed_dim, hidden_dim):
        super().__init__()
        self.hidden_dim = hidden_dim
        self.embedding = nn.Embedding(len(vocab), embed_dim)
        self.rnn1 = MyRNNCell(embed_dim, hidden_dim)
        self.rnn2 = MyRNNCell(hidden_dim, hidden_dim) #
        self.linear = nn.Linear(hidden_dim, 2)
        self.logsoftmax = nn.LogSoftmax(dim=0)

    def forward(self, sentence_tokens):
        self.rnn1.hidden_state = torch.zeros(self.hidden_dim)
        self.rnn2.hidden_state = torch.zeros(self.hidden_dim) #
        for one_token in sentence_tokens:
            one_embedded_token = self.embedding(one_token)
            self.rnn1(one_embedded_token)
            self.rnn2(self.rnn1.hidden_state) #

        feature_extractor_output = self.rnn2.hidden_state #
        class_scores = self.linear(feature_extractor_output)
        logprobs = self.logsoftmax(class_scores)
        return logprobs
```

שימו לב לכך שממד הקלט של התא הנשנה השני הוא כממד המצב החבוי, שכן המצב החבוי הראשון משמש כקלט עבורו.

במימוש הרשתות שלעיל הדגשנו את בהירות החישוב המבוצע ואת זרימת המידע על פני יעילות זמן הריצה. על כן, לפני אימון הרשת, נחליף את השכבות הנשנות שלנו בערימה (stack) של RNN מובנות של PyTorch, המבצעת חישוב זהה, אך בדרך יעילה יותר.

```
class FasterDeepRNNClassifier(nn.Module):
    def __init__(self, embed_dim, hidden_dim, RNNlayers):
        super().__init__()
        self.hidden_dim = hidden_dim
        self.embedding = nn.Embedding(len(vocab), embed_dim)
        self.rnn_stack = nn.RNN(embed_dim, hidden_dim, RNNlayers) #
        self.linear = nn.Linear(hidden_dim, 2)
        self.logsoftmax = nn.LogSoftmax(dim=0)

    def forward(self, sentence_tokens):
        all_embeddings = self.embedding(sentence_tokens)
        all_embeddings = all_embeddings.unsqueeze(1)
        hidden_state_history, _ = self.rnn_stack(all_embeddings)

        feature_extractor_output = hidden_state_history[-1, 0, :]
        class_scores = self.linear(feature_extractor_output)
        logprobs = self.logsoftmax(class_scores)
        return logprobs
```

ראו כי אנו מעבירים את כל הטוקנים של המשפט בבת אחת לערימת ה-RNN, וברקע מתבצע החישוב בטור אך לא בלולאת פייתון. הערימה מחזירה כפלט את היסטוריית המצבים החבויים של התא האחרון בערימה, התא השני במימוש הקודם שלנו, ואנו שולפים מטנזור זה את האיבר האחרון כפלט מחלף המאפיינים.

השכבות המובנות של PyTorch ערוכות לקבל minibatch של משפטים, כאשר בדרך כלל זהו הממד השני (הממד הראשון נשאר ממד הזמן), ולכן לפני הזנת המשפט אל שכבת ה-RNN המובנית הוספנו לטנזור השיכונים ממד מנוון בעזרת המתודה `unsqueeze`. הזנת הנתונים ב-batch דורשת עיבוד מקדים נוסף עקב אורכם המשתנה של המשפטים, ולא נדון בו בקורס זה. עם זאת, נוכל עדיין לאמן רשתות אלו כרגיל באמצעות הזנת המשפטים זה אחר זה, במחיר של תהליך אימון איטי יותר.

לרשתות נשנות יש יכולת לחזות רגשות שונים עבור משפטים בעלי אותם טוקנים כאשר אלו מופיעים במשפטים בסדר שונה, וזאת בשונה מהרשת המבוססת על סכום שיכוני הטוקנים. עם זאת, כדי להשיג מטרה זו עלינו לאמנן בהצלחה. זהו אתגר לא פשוט ונדון בסיבות לכך בפרק הבא.

שאלות לתרגול

1. אמנו RNN לחיזוי הרגש המובע במשפטים מאוסף הנתונים SST2 תוך כדי שימוש בסט אימון קטן מאוד, עד שתתקבל בבירור התאמת יתר. הראו זאת באמצעות ציור גרף של שגיאת האימון ושגיאת הבדיקה.
2. א. מחקו את השורה הראשונה במתודה `forward()` של רשת מסוג `RNNClassifier`, שבה מאפסים את המצב החבוי לפני הזנת המשפט לרשת הנשנית. כעת חזרו על השאלה הקודמת והשוו את התוצאות.
ב. הסבירו את נחיצות האיפוס של המצב החבוי.
3. אמנו RNN עמוקה ותוך כדי האימון, עבור כל פרמטר ברשת, שמרו בנפרד את נורמת אינסוף של הגרדיאנט שלו לאחר כל איטרציה של לולאת האימון. ציירו גרף של ממוצע הנורמות כפונקציה של `epoch`.

התפשטות לאחור דרך הזמן

בעת הפעלת אלגוריתם אופטימיזציה מבוסס גרדיאנט על רשת נשנית, לא ניכר על פניו שתתעורר בעיה: כל פעולות החשבון שאנו משתמשים בהן לחישוב המצב החבוי האחרון הן גזירות, ולכן ניתן לחשב דרכן לאחור את גרדיאנט פונקציית השגיאה. ואכן כך פועלת מערכת ה-autograd של PyTorch. עם זאת, המבנה המסוים של רשת נשנית, שבו אנו משתמשים באותם פרמטרים שוב ושוב בשלבים שונים בחישוב, מוביל לבעיות של יציבות נומרית. נדון בהן כעת.

זכרו כי נוסחת עדכון המצב החבוי של תא נשנה פשוט היא

$$h_{t+1} = \tanh(W_{input}X_t + b_{input} + W_{hidden}h_t + b_{hidden})$$

אך כעת חשבו על כך שעבור משפט קלט באורך T טוקנים אנו מפעילים נוסחה זו T פעמים, עד ליצירת המצב החבוי הסופי, כפי שהופיע באיור הרשת הפרוסה בפרק הקודם.

נתחיל את הדיון במקרה הפשוט שבו המצב החבוי הוא חד־ממדי: h_t הוא סקלר, ולפיכך W_{hidden} מטריצת משקלי השכבה הלינארית החבויה, מתנוונת לסקלר גם היא. נסמן אפוא $W_{hidden} = w$. עתה נרצה לחשב את נגזרת פונקציית המחיר לפי פרמטר זה, $\frac{\partial C}{\partial w}$, לצורך ביצוע צעד העדכון של SGD.

בהתפשטות לפנים, w משפיע על החישוב המבוצע ברשת בפעם האחרונה בעת חישוב המצב החבוי האחרון, h_T , ולכן אם נניח כי הנגזרת $\frac{\partial C}{\partial h_T}$ כבר חושבה, נקבל מכלל השרשרת:

$$\frac{\partial C}{\partial w} = \frac{\partial C}{\partial h_T} \frac{\partial h_T}{\partial w}$$

בבואנו לחשב את

$$\frac{\partial h_T}{\partial w} = \frac{\partial}{\partial w} \tanh(W_{input}X_t + b_{input} + wh_{T-1} + b_{hidden})$$

נפגוש באתגר שנוצר עקב המבנה הרקורסיבי של הרשת: מצד אחד, w מופיע מפורשות בנוסחת המעבר, ומצד אחר גם h_{T-1} עצמו חושב בעזרת אותם פרמטרים, וחישוב זה משפיע אף הוא על הגרדיאנט. על כן, יש:

1. לחשב את הנגזרת המיידית $\frac{\partial h_T}{\partial w}$ כאילו h_{T-1} קבוע. נסמן אותה ב- $\left[\frac{\partial h_T}{\partial w}\right]$ למניעת בלבול.

2. לחשב לאחור, לזמן הקודם, את הנגזרת, כלומר לחשב את $\frac{\partial h_T}{\partial h_{T-1}}$.

3. לחשב את $\frac{\partial h_{T-1}}{\partial w}$.

4. לחבר את התוצאות בעזרת כלל השרשרת.

נקבל לבסוף,

$$\frac{\partial h_T}{\partial w} = \left[\frac{\partial h_T}{\partial w}\right] + \frac{\partial h_T}{\partial h_{T-1}} \frac{\partial h_{T-1}}{\partial w}.$$

נוכל לחשב את $\left[\frac{\partial h_T}{\partial w} \right]$ ו- $\frac{\partial h_T}{\partial h_{T-1}}$ בנקל, ונעשה זאת אחר כך. לעומת זאת, בבואנו לחשב את $\frac{\partial h_{T-1}}{\partial w}$ תתעורר אותה בעיה כמקודם, שכן גם h_{T-1} תלוי ב- w באותן שתי דרכים: w מופיע בנוסחת המעבר מ- h_{T-2} אל h_{T-1} , וגם h_{T-2} עצמו חושב בעזרת w . על כן יש להמשיך ולחשב לאחור את השגיאה אל h_{T-2} , בדומה לחישוב שביצענו לעיל:

$$\frac{\partial h_{T-1}}{\partial w} = \left[\frac{\partial h_{T-1}}{\partial w} \right] + \frac{\partial h_{T-1}}{\partial h_{T-2}} \frac{\partial h_{T-2}}{\partial w}$$

נציב ביטוי זה בנוסחה עבור $\frac{\partial h_T}{\partial w}$ ונקבל:

$$\begin{aligned} \frac{\partial h_T}{\partial w} &= \left[\frac{\partial h_T}{\partial w} \right] + \frac{\partial h_T}{\partial h_{T-1}} \left(\left[\frac{\partial h_{T-1}}{\partial w} \right] + \frac{\partial h_{T-1}}{\partial h_{T-2}} \frac{\partial h_{T-2}}{\partial w} \right) = \\ &= \left[\frac{\partial h_T}{\partial w} \right] + \frac{\partial h_T}{\partial h_{T-1}} \left[\frac{\partial h_{T-1}}{\partial w} \right] + \frac{\partial h_T}{\partial h_{T-1}} \frac{\partial h_{T-1}}{\partial h_{T-2}} \frac{\partial h_{T-2}}{\partial w}. \end{aligned}$$

השיקולים הנ"ל תקפים עבור כל אחד מהמצבים החבויים הקודמים, ולכן יהיה צורך להמשיך ולחשב את השגיאה לאחור דרך הזמן עד שנגיע ל- h_1 , כך שלבסוף הנוסחה שלפיה יש לחשב את $\frac{\partial h_T}{\partial w}$ היא:

$$\begin{aligned} \frac{\partial h_T}{\partial w} &= \left[\frac{\partial h_T}{\partial w} \right] + \\ &+ \frac{\partial h_T}{\partial h_{T-1}} \left[\frac{\partial h_{T-1}}{\partial w} \right] + \\ &+ \frac{\partial h_T}{\partial h_{T-1}} \frac{\partial h_{T-1}}{\partial h_{T-2}} \left[\frac{\partial h_{T-2}}{\partial w} \right] + \\ &\vdots \\ &+ \frac{\partial h_T}{\partial h_{T-1}} \frac{\partial h_{T-1}}{\partial h_{T-2}} \dots \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial w} \end{aligned}$$

בנוסחה זו יש שורה עבור כל אחת מאיטרציות הלולאה המופיעה בחישוב ההתפשטות לפנים ברשת אלמן. השורה הראשונה היא התרומה של האיטרציה האחרונה, הקולטת את הטוקן האחרון במשפט, בעוד השורה האחרונה בנוסחה היא התרומה של האיטרציה הראשונה. הבעייתיות החשובית בנוסחה זו, מעבר לאורכה, נגלית לעין כאשר שמים לב ש-

$$\frac{\partial h_t}{\partial h_{t-1}} = (1 - h_t^2) \left[\frac{\partial}{\partial h_{t-1}} (W_{input} X_t + b_{input} + w h_{t-1} + b_{hidden}) \right] = (1 - h_t^2) w$$

שכן $\tanh'(x) = 1 - \tanh^2(x)$. בהצבת תוצאה זו בנוסחה שלעיל נקבל

$$\begin{aligned} \frac{\partial h_T}{\partial w} &= \left[\frac{\partial h_T}{\partial w} \right] + \\ &+ w(1 - h_T^2) \left[\frac{\partial h_{T-1}}{\partial w} \right] + \\ &+ w^2(1 - h_T^2)(1 - h_{T-1}^2) \left[\frac{\partial h_{T-2}}{\partial w} \right] + \\ &\vdots \\ &+ w^{T-1}(1 - h_T^2)(1 - h_{T-1}^2) \dots (1 - h_2^2) \frac{\partial h_1}{\partial w} \end{aligned}$$

כעת ניכרים הגורמים הבעייתיים בנוסחה: הם מהצורה w^t . אם ערך הפרמטר הנוכחי גדול מ-1 (בערכו המוחלט), גורמים אלו שואפים במהרה לאינסוף, כך שעבור משפט קלט ארוך דיו הנגזרת $\frac{\partial h_T}{\partial w}$ תהיה גדולה מאוד בערכה המוחלט, דבר אשר ימנע מאלגוריתם האופטימיזציה להתכנס – זוהי תופעת הגרדיאנט המתפוצץ (exploding gradient).

עבור $|w| < 1$ המצב אינו טוב בהרבה: גורמים אלו ישאפו במהרה לאפס, ויחד איתם גם ההשפעה של הטוקנים המופיעים בתחילת המשפט על $\frac{\partial h_T}{\partial w}$. עקב כך ערך נגזרת זו יחושב בעיקרו על סמך התרומות של הטוקנים האחרונים שהוזנו לרשת, והרשת לא תוכל ללמוד תלויות ארוכות טווח בתוך המשפט.

מכיוון שקשיים אלו נובעים מההתפשטות לאחור דרך המצבים החבויים, הם רלוונטיים גם עבור שאר פרמטרי הרשת המופיעים לפני התא הנשנה, שכן עבורם יש לחשב לאחור את הנגזרות דרך התא. קשיים אלו מתעוררים גם כאשר המצב החבוי הוא רב-ממדי, מאחר שאת הביטויים w^t מחליפות חזקות של המטריצה W_{hidden} , אשר להן התנהגות אסימפטוטית דומה.

ישנם כלים אחדים להתמודד עם הקושי המובנה שיוצרת התפשטות הגרדיאנט דרך רשת נשנית. חלקם מנסים לפתור את הבעיה באמצעות שינוי אלגוריתם האופטימיזציה, למשל בעזרת הקטנת קצב הלמידה כאשר הגרדיאנט גדול מדי, בדומה לאלגוריתמים בעלי קצב למידה אדפטיבי שלמדנו עליהם ביחידה 3.

הכלים המוצלחים יותר פותרים את הבעיה באמצעות שינוי ארכיטקטורת הרשת: במקום תא אלמן פשוט, הרשת תשתמש בתאים נשנים מתקדמים, המכילים רכיבים דומים לחיבורי הדילוג של רשתות שיריות. הנפוץ בהם הוא תא LSTM (Long Short-Term Memory), אשר מכיל גם רכיב בקרת זרימה המאפשר לתא ללמוד מתי לפתוח ומתי לסגור את חיבור הדילוג, וכן רכיב נלמד המאפשר לתא לאפס את המצב החבוי, למשל לאחר שהוא זיהה נתק סמנטי בין שני חלקי משפט.

רשתות מבוססות LSTM מומשו ב-PyTorch, והפלט שלהן זהה במבנהו לזה של רשת RNN פשוטה. על כן נוכל להשתמש בהן בנקל, כדלהלן:

```
class LSTMClassifier(FasterDeepRNNClassifier):
    def __init__(self, embed_dim, hidden_dim, RNNlayers):
        super().__init__(embed_dim, hidden_dim, RNNlayers)
        self.rnn_stack = nn.LSTM(embed_dim, hidden_dim, RNNlayers)
```

ראו כיצד רשת הסיווג החדשה יורשת מהרשת האחרונה שהגדרנו בפרק הקודם, שבה `rnn_stack` הוא אובייקט מסוג `nn.LSTM`. השינוי היחיד שיש לעשות הוא להגדירו מחדש כ-`LSTM`.

לסיום דיון זה, נציין כי בעוד רשתות LSTM (או רשתות בעלות תאים נשנים דומים) הפגינו את הביצועים הטובים ביותר במשימות עיבוד שפה טבעית עד לשנת 2017, נכון לכתיבת שורות אלו בשנת 2022, רשתות מבוססות transformer הן המוצלחות ביותר במשימות אלו. ארכיטקטורת ה-`transformer`, שבה נדון ביחידת הלימוד הבאה, אינה נשנית ולכן מתחמקת משני האתגרים המרכזיים של שימוש ב-`RNN`: איטיות החישוב בטור ובעיות הגרדיאנט המתפוצצ/הנעלם.

שאלות לתרגול

1. חשבו את $\left[\frac{\partial h_t}{\partial w} \right]$ במפורש, והראו כי ביטוי זה חסום.

2. א. הראו כי קיים חסם K כך שלכל t מתקיים

$$\left\| \left(1 - h_t^2\right) \left(1 - h_{t-1}^2\right) \cdots \left(1 - h_1^2\right) \left[\frac{\partial h_{t-1}}{\partial w} \right] \right\| < K$$

ב. על סמך הסעיף הקודם, מצאו חסם ל- $\frac{\partial h_t}{\partial w}$ התלוי ב- w דרך גורמים מהצורה w^t בלבד.

3. חשבו את $\frac{\partial h_1}{\partial w}$ בהנחה ש- h_0 הוא וקטור האפס, כנהוג באתחול תא נשנה. הסבירו מדוע לא מופיע

ביטוי מהצורה $\left[\frac{\partial h_1}{\partial w} \right]$ לעיל.

יחידה 7:

ארכיטקטורות מקודד-מפרש

מקודד עצמי

עד כה ראינו בקורס ארכיטקטורות רשת שונות, והשתמשנו בהן כדי לחלץ מאפיינים מועילים למטרת הרשת, אשר הייתה פתרון לבעיות רגרסיה או סיווג. למטרה זו, חיברנו בסוף כל רשת ראש סיווג המורכב משכבה לינארית ופונקציית softmax או ראש רגרסיה המורכב משכבה לינארית בלבד. ביחידה זו נתמודד עם משימות מגוונות, כגון הפחתת ממדים, דחיסת מידע ותרגום טקסט, ולפיכך ראש הרשת שנבחר יהיה מתוחכם יותר ומותאם למשימה הנדרשת.

המבנה הבסיסי של רשת המסוגלת לפתור את הבעיות הנ"ל הוא בתצורת מקודד-מפרש (encoder-decoder). תפקידו של המקודד הוא כמחלץ המאפיינים ברשתות המוכרות לנו: עליו לייצר ייצוג יעיל של הנתונים עבור משימת ההמשך. המפרש מחליף את ראש הסיווג או את ראש רגרסיה, ותפקידו לתרגם את פלט המקודד, הקרוי לעתים גם הייצוג החבוי (latent representation) לפלט הרצוי מהרשת. למשל, עבור רשת המיועדת לתרגום משפט מאנגלית לעברית, המקודד ייצר ייצוג חבוי של המשפט באנגלית, וקטור של מספרים ממשיים בממד קבוע, בעוד המפרש יתרגם ייצוג חבוי זה למשפט בעברית.

כדוגמה ראשונה לארכיטקטורה זו, נעסוק במקודד עצמי (autoencoder) אשר ישמש אותנו להקטנת ממד ולדחיסת מידע. פלט הרשת יהיה זהה ככל האפשר לקלט הרשת, אך ממד המרחב החבוי (latent space) יהיה קטן מממד הקלט, כך שהנתונים יידרשו לעבור דרך "צוואר בקבוק". לאחר תהליך אימון מוצלח, עם פונקציית מחיר מתאימה, נצפה שהמקודד ילמד לייצג את הנתונים בצורה תמציתית, תוך כדי אובדן מידע מינימלי, שכן המפרש בתורו נדרש לתרגם את הייצוג החבוי חזרה לקלט המקורי.

לצורך הדגמת רעיון זה נחזור לאוסף הנקודות השחורות והלבנות, הדוגמה הראשונה שעסקנו בה ביחידה 2. כזכור, דגמנו 100 נקודות במרחב דו-ממדי, ושמרנו את הקואורדינטות בטנזור X . כעת נבנה מקודד עצמי פשוט ביותר אשר יקבל כקלט טנזור זה.

```
encoder = nn.Linear(2,1)
decoder = nn.Linear(1,2)
autoencoder = nn.Sequential(encoder,decoder)
```

ראו כי המקודד מורכב משכבה לינארית יחידה, המקבלת כקלט את שתי הקואורדינטות של הדגימות ב־ X ומחזירה פלט חד-ממדי. המפרש ישחזר את הנתונים לממד המקורי בשכבה לינארית אחרת.

ברצוננו לאמן רשת זו לבצע משימתה ככל רשת אחרת, בעזרת אלגוריתם מבוסס גרדיאנט. לשם כך אנו נדרשים להגדיר את פונקציית המחיר. מכיוון שנרצה שפלט המפרש יהיה קרוב ככל האפשר לקלט הרשת, ריבוע המרחק האוקלידי ביניהם הוא בחירה טבעית. אם כן, נסמן את קלט הרשת (נקודת

דגימה יחידה) ב־ (x_0, x_1) ואת הפלט המתאים ב־ (y_0, y_1) , ונקבל שהתרומה למחיר הכולל של נקודה זו היא:

$$H(x_0, x_1) = \|(y_0, y_1) - (x_0, x_1)\|^2 = (y_0 - x_0)^2 + (y_1 - x_1)^2$$

כרגיל, המחיר הכולל יהיה ממוצע של ערך זה עבור כל הדגימות, ונממש אותו בקוד בעזרת הפונקציה `nn.MSELoss`, המוכרת לנו מרשתות הרגרסיה. חישוב פונקציית המחיר בתוך לולאת האימון יתבצע כך:

```
reconstructed_X = autoencoder(X)
loss             = MSELoss(reconstructed_X, X)
```

ראו כי הסיווג הנתון של הנקודות (שחורות או לבנות) אינו משתתף בתהליך האימון, ותנו דעתכם לכך שתהליך אימון המקודד העצמי מבוצע באופן בלתי מפוקח.

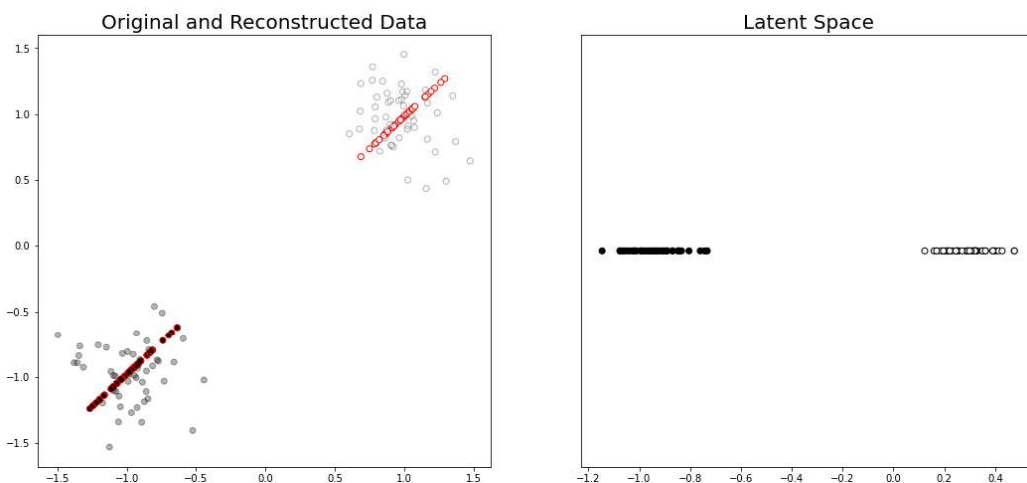
לאחר האימון נוכל לפצל את המקודד העצמי לחלקיו, להזין למקודד **בלבד** את הנתונים, ולקבל ייצוג חבוי של הנתונים במרחב חד־ממדי.

```
with torch.no_grad():
    encoded_X = encoder(X)
    encoded_X.size()

    torch.Size([100, 1])
```

פלט:

מובן שמעבר לייצוג חבוי זה כרוך באובדן מידע על הנתונים המקוריים, שכן פלט המקודד העצמי בהכרח יושב על קו ישר יחיד, כפי שניכר מהאיור שלהלן שבו המידע המשוחזר מסומן בנקודות בעלות שוליים אדומים.



עבור משימות למידה מסוימות אובדן המידע דווקא רצוי, למשל כאשר נרצה לאמן בהמשך מודל המבצע הכללה על סמך אוסף נתוני האימון. צוואר הבקבוק מאלץ את המקודד להיפטר ממידע הספציפי לאוסף האימון (בין השאר), וההכללה על סמך הייצוג החבוי לעתים פשוטה יותר. גישה זו שימושית בייחוד כאשר קיים סט נתונים בלתי מפותח גדול, ורק חלקו הקטן מתויג. על אוסף הנתונים כולו נוכל לאמן מקודד עצמי, ואחרי כן נוכל לאמן מודל סיווג פשוט על הנתונים המתויגים, כאשר הקלט למודל הסיווג יהיה הייצוג החבוי המתקבל מהמקודד.

שאלות לתרגול

1. כתבו במפורש את החישוב המתבצע במקודד העצמי הנ"ל, והוכיחו שפלט הרשת כפונקציה של הקלט (x_0, x_1) הוא קו ישר.
רמז: הציגו את y_0, y_1 כפונקציות של x_0, x_1 ומצאו הצגה של y_1 כפונקציה של y_0 . אפשר להניח שפרמטרים מסוימים אינם מתאפסים.
2. א. חברו ראש סיווג לפלט המקודד המאומן ואמנו את ראש הסיווג בלבד להבדיל בין נקודות שחורות ללבנות על סמך הייצוג החבוי שלהן. השוו את המודל המתקבל למודל הניירון היחיד שאימנו ביחידה 2.
ב. כעת חזרו על אימון שתי הרשתות מסעיף א, אך הפעם השתמשו בסט אימון המורכב רק מנקודה שחורה יחידה ונקודה לבנה יחידה. בדקו את ביצועיהם על שאר הנקודות.
ג. באיזה מסווג תעדיפו להשתמש עבור נתונים חדשים?

שימושים נוספים של מקודדים עצמיים

עקרון הפעולה של המקודד העצמי שהכרנו בפרק הקודם תקף גם כאשר אוסף הנתונים המוזן אליו מורכב יותר מאשר נקודות במרחב דו־ממדי. בפרק זה נאמן מקודדים עצמיים לדחיסת תמונות מאוסף הנתונים Fashion-MNIST ולשחזור תמונות מושחתות, כדוגמה לשניים מהשימושים האפשריים של ארכיטקטורה זו.

בשלב הראשון, נאפשר למקודד ולמפרש ללמוד תלויות לא לינאריות בעזרת תוספת של פונקציות אקטיבציה לאחר האגרסיה הלינארית. ראו זאת בקטע הקוד שלהלן, ושימו לב לכך שבשלב ראשון אנו משטחים את טנזור הקלט, שכן batch של N תמונות מאוסף הנתונים מגיע כטנזור בגודל $N \times 1 \times 28 \times 28$, בעוד השכבה הלינארית מצפה לקלט בממדים $N \times 784$.

```
class Encoder(nn.Module):
    def __init__(self, latent_dim):
        super().__init__()
        self.linear = nn.Linear(784, latent_dim)
        self.relu = nn.ReLU()
    def forward(self, image):
        flattened = image.flatten(start_dim=1)
        compressed_image = self.relu(self.linear(flattened))
        return compressed_image
```

בבואנו לכתוב את המפרש, עלינו לתת את הדעת לצורת הפלט הרצויה ממנו. ברצוננו לשחזר את התמונות המוזנות אל הרשת, אשר בזמן טעינתן עברו נרמול: ערך כל פיקסל הוא מספר ממשי בטווח בין 0 ל־1. על כן נשתמש באקטיבציית הסיגמואיד המייצרת פלט בטווח הדרוש. כמו כן, עלינו להפוך את פעולת השיטוח, ולהחזיר פלט בממדים הזהים לאלו של קלט המקודד. ראו כיצד אילוצים אלו באים לידי ביטוי בקוד. שימו לב במיוחד לפרמטים של מתודת ה־`reshape`.

```
class Decoder(nn.Module):
    def __init__(self, latent_dim):
        super().__init__()
        self.linear = nn.Linear(latent_dim, 784)
        self.sigmoid = nn.Sigmoid()
    def forward(self, compressed_image):
        decoded = self.linear(compressed_image)
        decoded = self.sigmoid(decoded)
        reconstructed_image = decoded.reshape(-1, 1, 28, 28)
        return reconstructed_image
```

כעת נוכל לבחור ערך קטן עבור ממד המרחב החבוי, לחבר בטור מקודד ומפרש, ולאמנם יחדיו כמקודד עצמי. למשל, בשורה שלהלן אנו מגדירים מקודד עצמי בעל מרחב חבוי עשר־ממדי (זכרו שממד הקלט הוא 28×28).

```
autoencoder = nn.Sequential(Encoder(10), Decoder(10))
```

לצורך אימון המקודד העצמי נעביר דרכו batch של תמונות ונשווה את התוצאה לתמונות המקוריות. פונקציית ההפסד תהיה שוב `nn.MSELoss`, כאשר בהקשר הנוכחי היא קרויה "מחיר השחזור" (reconstruction loss), מסיבות ברורות. תוצאות אימון מוצלח מופיעות באיור המצורף, שבו מוצגים המקור ופלט המקודד העצמי של זוג תמונות מאוסף נתוני הבדיקה.



ראו כי השחזור אינו מושלם, שכן הנתונים עברו דרך צוואר בקבוק אשר גודלו כ-13% מגודל הנתונים המקוריים.

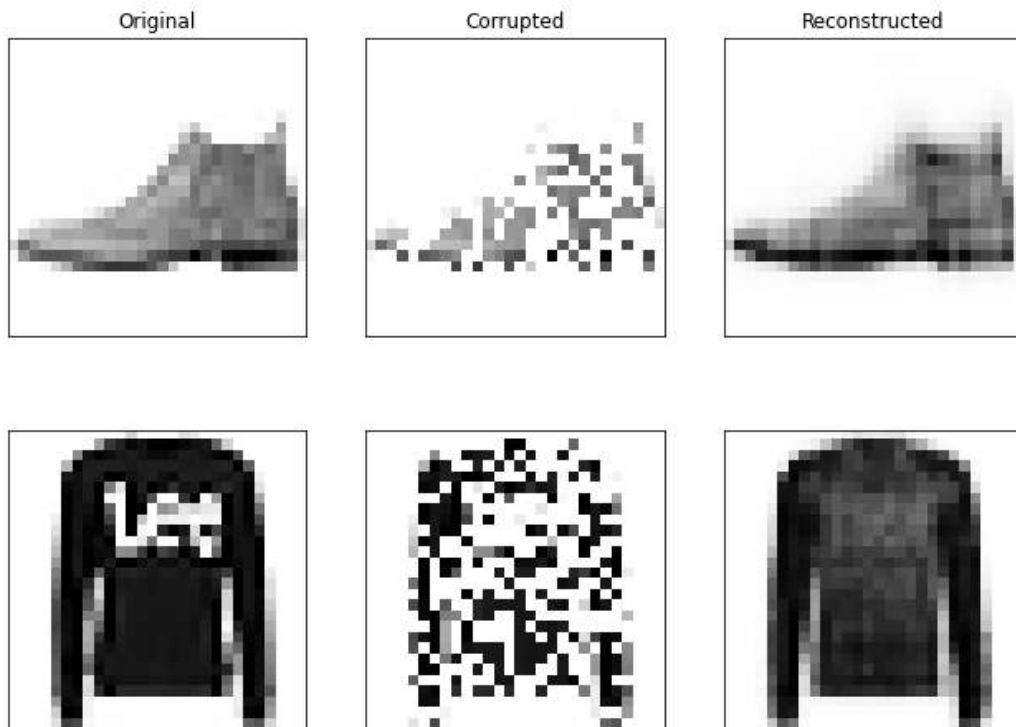
אם יהיה ברצוננו להשתמש במקודד לצורך משימות המשך, למשל עבור אימון מסווג על סמך הייצוג החבוי, נוכל לחלץ אותו בנקל, שכן הוא המודול הראשון באובייקט `autoencoder`.

```
encoder = autoencoder[0]
```

כעת על בסיס הפלט של האובייקט encoder, נוכל לבנות רשת סיווג חדשה ולאמן בה את הפרמטרים של השכבות החדשות בלבד.

מקודד עצמי לניקוי רעש

נוכל להשתמש באותה ארכיטקטורת רשת שלעיל גם לצורך אימון מקודד עצמי המקבל תמונות אשר עברו השחתה ומשחזר אותן. רשת המשמשת למטרה זו נקראת באנגלית Denoising Auto-Encoder (DAE). לפני שנדון בפרטי המימוש, ראו באיור שלהלן שחזורים של DAE מאומן. הקלט לרשת הוא התמונה האמצעית, אשר חצי מהפיקסלים שלה אופסו באקראי. הרשת מחזירה את הפלט מימין, מבלי שנחשפה מעולם לתמונה המקורית (משמאל).



דרושים שינויים ספורים בבניית המקודד העצמי ואימונו כדי לקבל תוצאה זו. ראשית, נרצה ליצור יתירות במרחב החבוי, כדי להתמודד עם איהוודאות המובנית בנתונים רועשים. לצורך כך נבחר ממד מרחב חבוי הגדול במעט מממד הנתונים המקוריים.

```
latent_dim = int(784 * 1.2)
DAE = nn.Sequential(Encoder(latent_dim), Decoder(latent_dim))
```

שנית, עלינו להשחית את הנתונים למטרות האימון. לצורך כך, לאחר טעינת batch של תמונות למשתנה original_imgs, נעביר אותו דרך שכבת dropout אשר תאפס אקראית חצי מהפיקסלים בכל תמונה.

```
corruptor = nn.Dropout()
corrupted_imgs = corruptor(original_imgs)
```


לבסוף, בתוך לולאת האימון נזכור להזין לרשת את התמונות המושחתות, אך את הפלט נשווה לתמונות המקוריות. מחיר השחזור יתגמל מודלים אשר ממלאים את החסר בתמונות המושחתות, כפי שאנו מעוניינים.

```
reconstructed = DAE(corrupted_imgs)
loss          = MSELoss(reconstructed, original_imgs)
```

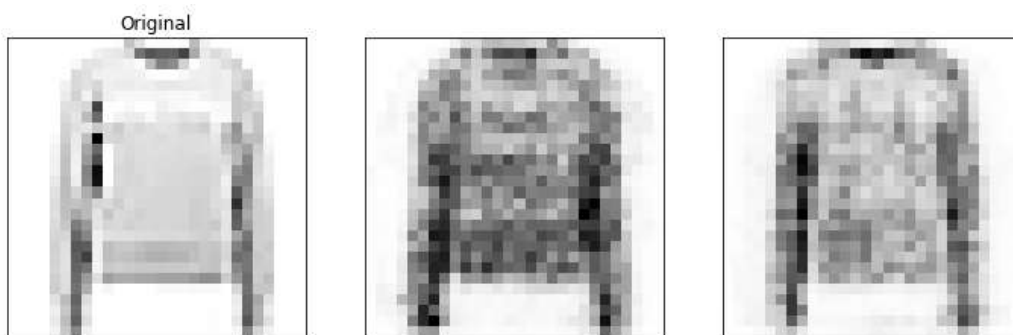
לאחר שינוי זה כל שנותר לעשות הוא להפעיל את אלגוריתם האימון.

דגימת נתונים חדשים

שימוש נוסף של המקודד העצמי הוא למטרת ייצור דגימות מלאכותיות חדשות הדומות לתמונות הקלט. לאחר האימון, אנו יכולים להזין ייצוג חבוי חדש לתוך **המפרש** המאומן, ולקבל תמונה משוחזרת המתאימה לייצוג חבוי זה, וזאת אף על פי שהוא אינו מייצג אף תמונה מקורית. ראו דוגמה לכך בקטע הקוד שלהלן.

```
encoder = DAE[0]
decoder = DAE[1]
num_samples = 2
with torch.no_grad():
    encoded          = encoder(one_img)
    perturbation      = torch.randn((num_samples, *encoded.size()))
    perturbed_latent = encoded * (1 + perturbation)
    new_samples      = decoder(perturbed_latent)
```

ראו כיצד אנו מוסיפים לקידוד של תמונה מקורית רעש אקראי ומכניסים את התוצאה למפרש. שתי דגימות חדשות המתקבלות בדרך זו, לצד תמונת המקור, מופיעות באיור שלהלן.



בדוגמה זו אנו פוגשים לראשונה ברשת נוירונים **המייצרת** נתונים חדשים. רשתות המשמשות למטרות אלו הן **מודלים גנרטיביים**, וקיימות עבורן ארכיטקטורות רשת מוצלחות במיוחד אשר מפגינות יצירתיות יוצאת דופן לא רק במשימות הקלאסיות של למידת מכונה ומדעי הנתונים, אלא גם ביצירת אומנות ויזואלית. נושא זה הוא תחום מחקר ער ופורה, אך בקורס הנוכחי לא נעסוק בו מעבר להיכרות הבסיסית שערכנו זה עתה.

ככל רשת נוירונים, גם מקודדים עצמיים יוכלו ללמוד ייצוגים חבויים מוצלחים באמצעות העמקת הרשת. בבואנו להוסיף שכבות למקודד לא תתעורר בעיה, ולמעשה נוכל אפילו להשתמש ברשת שאומנה למטרה אחרת, כגון ResNet, להוריד לה את ראש הסיווג ולהשתמש במחלף המאפיינים שלה בתור המקודד. עם זאת, עבור המפרש נדרש לבצע פעולה הפוכה: עיבוד נתונים הדרגתי בחזרה מהייצוג החבוי לתמונה. עם אתגר זה נתמודד בפרק הבא.

שאלות לתרגול

1. אמנו כמה מקודדים עצמיים לניקוי רעש על אוסף הנתונים Fashion-MNIST הנבדלים זה מזה בגודל המרחב החבוי. השוו את התוצאות ומצאו את הערך האופטימלי. האם מצאתם DAE מוצלח בעל ממד חבוי הקטן מממד הנתונים המקוריים? הסבירו תוצאה זו על סמך יתירות המידע באוסף הנתונים עצמו.
2. חזרו על השאלה הקודמת עם אוסף הנתונים MNIST. מה תוכלו להגיד על ההבדל בין אוספי הנתונים?
הערות:
 1. כדי לטעון את MNIST השתמשו בפונקציה `torchvision.datasets.MNIST`.
 2. שימו לב שאוספי הנתונים זהים מכל הבחינות (גודל התמונה, מספר דגימות, מספר מחלקות וכו'), מלבד תוכן התמונות עצמן.

קונבולוציה משוחלפות ושימושיה

שכבות הקונבולוציה שהשתמשנו בהן עד כה מקבלות כקלט תמונה בגודל $H \times W$ בעלת מספר ערוצים כלשהו, ולרוב מקטינות את ממד האורך והרוחב תוך כדי הגדלת מספר הערוצים, או לכל היותר שומרות על הגודל המקורי בעזרת ריפוד תמונת הקלט. היזכרו למשל במבנה הרשת ResNet, אשר שכבות הקונבולוציה שלה לעתים שומרות את ממד פלט זהה לממד הקלט, ולעתים חוצות את ממד האורך וממד הרוחב. שכבות אלו שימשו פתרון מצוין למטרת חילוץ מאפיינים אשר קושרים בהדרגה בין אזורים שונים בתמונה המקורית. עתה אנו רוצים לתכנן מפרש המקבל מאפיינים אלו מהמקודד ומשחזר את התמונה המקורית, ולשם כך עלינו לפעול בכיוון ההפוך: הגדלת ממד האורך והרוחב, תוך כדי הקטנת מספר הערוצים. לשם כך נשתמש בפעולה חדשה: הקונבולוציה המשוחלפת.

אנו מעוניינים בפעולה המקבלת כקלט תמונה בגודל $H \times W$ בעלת C ערוצים, והפלט שלה הוא תמונה **גדולה יותר** בממד האורך והרוחב, עם מספר ערוצים **קטן יותר**. לאור ההצלחה של שכבות קונבולוציה בעיבוד מידע ויזואלי, נרצה לשמור על תכונותיהן היסודיות גם בשכבות המבצעות את הפעולה ההפוכה, ובפרט:

- על השכבה להשתמש במספר קטן של פרמטרים נלמדים, ויהיה עליה לבצע שימוש חוזר בהם, בדומה לגרעין של שכבת קונבולוציה.
- הפעולה תתבצע מקומית: פיקסל בתמונת הפלט יחושב על סמך כמה פיקסלים הסמוכים זה לזה בתמונת הקלט בלבד.

בבחינה מעמיקה של תהליך האימון של רשתות קונבולוציה המוכרות לנו, נוכל להבין שמבלי לשים לב אנו כבר משתמשים בפעולה העונה על כל הדרישות הנ"ל, והיא התפשטות הגרדיאנט לאחור דרך שכבת קונבולוציה. נדגים זאת בעזרת שכבת קונבולוציה המקבלת כקלט תמונה בגודל $H \times W$ בעלת ערוץ יחיד ומבצעת קונבולוציה שלה עם גרעין K בגודל $p \times q$. בדומה לדיון שערכנו בעת הצגת שכבות הקונבולוציה, נסמן את תמונת הקלט ב-

$$X = \begin{pmatrix} x_{1,1} & \cdots & x_{1,W} \\ \vdots & \ddots & \vdots \\ x_{H,1} & \cdots & x_{H,W} \end{pmatrix}$$

את הגרעין ב-

$$K = \begin{pmatrix} k_{1,1} & \cdots & k_{1,q} \\ \vdots & \ddots & \vdots \\ k_{p,1} & \cdots & k_{p,q} \end{pmatrix}$$

ואת תמונת הפלט ב-

$$Y = \begin{pmatrix} y_{1,1} & \cdots & y_{1,W_y} \\ \vdots & \ddots & \vdots \\ y_{H_y,1} & \cdots & y_{H_y,W_y} \end{pmatrix}$$

זכרו שהממד של תמונת הפלט תלוי בממדי תמונת המקור והגרעין, כך ש- $H_y = H - p + 1$, וכן $W_y = W - q + 1$.

כעת נניח שבעת ההתפשטות לאחר חישובנו זה מכבר את גרדיאנט פונקציית המחריר לפי פלט השכבה. כלומר ערכי המטריצה

$$\frac{\partial C}{\partial Y} = \begin{pmatrix} \frac{\partial C}{\partial y_{1,1}} & \dots & \frac{\partial C}{\partial y_{1,W_y}} \\ \vdots & \ddots & \vdots \\ \frac{\partial C}{\partial y_{H_y,1}} & \dots & \frac{\partial C}{\partial y_{H_y,W_y}} \end{pmatrix}$$

ידועים, וברצוננו לחשב את $\frac{\partial C}{\partial X}$.

התובנות העיקריות שיש להסיק מהחישוב העתיד לבוא כעת הן:

- פעולת התפשטות הגרדיאנט לאחר מקבלת כקלט "תמונה" קטנה $\frac{\partial C}{\partial Y}$, וממדיה כממד Y .
- היא מחזירה כפלט "תמונה" גדולה $\frac{\partial C}{\partial X}$, מממד $H \times W$.
- כל זאת תוך כדי שימוש חוזר בגרעין הקונבולוציה ושמירה על אינטראקציות לוקליות.

ניגש אם כן למשימה. כזכור, איברי Y חושבו באמצעות חיתוך תת-תמונות קטנות מתמונת המקור, חישוב המכפלה איבר-איבר עם הגרעין וסכימת התוצאה, ובנוסחה:

$$y_{r,s} = \sum_{i=1}^p \sum_{j=1}^q x_{r+i-1,s+j-1} k_{i,j}$$

מנוסחה זו ניכר מייד כי:

$$\frac{\partial y_{r,s}}{\partial x_{r+i-1,s+j-1}} = k_{i,j} \quad i=1,\dots,p, \quad j=1,\dots,q$$

או במילים: לכל פיקסל $x_{n,m}$ בתמונת המקור אשר השתתף בחישוב $y_{r,s}$, הנגזרת $\frac{\partial y_{r,s}}{\partial x_{n,m}}$ היא איבר הגרעין אשר כפל את $x_{n,m}$ בעת חישוב פיקסל הפלט. מובן שלכל $x_{n,m}$ אשר לא השתתף בחישוב $y_{r,s}$ מתקיים $\frac{\partial y_{r,s}}{\partial x_{n,m}} = 0$. נשלב תוצאה זו עם כלל השרשרת, שלפיו

$$\frac{\partial C}{\partial x_{n,m}} = \sum_{r=1}^{H_y} \sum_{s=1}^{W_y} \frac{\partial C}{\partial y_{r,s}} \frac{\partial y_{r,s}}{\partial x_{n,m}}$$

ונקבל שכדי לחשב את $\frac{\partial C}{\partial x_{n,m}}$ יש לעבור על כל הפיקסלים $y_{r,s}$ אשר בחישובם השתתף $x_{n,m}$, לכפול

את $\frac{\partial C}{\partial y_{r,s}}$ באיבר הגרעין המתאים ל- $x_{n,m}$ בחישוב זה, ולסכום את כל התוצאות. בקטע הקוד שלהלן

אנו ממשים רעיון זה. שימו לב כיצד אנו עוברים על כל פיקסל $y_{r,s}$, "נוזכרים" בחישוב שביצענו בהתפשטות לפנים ובהתאם לכך מוסיפים איברים לגרדיאנט הנצבר במשתנה dC/dX . ראו גם כי אנו עושים זאת בזימנית עבור כל הפיקסלים אשר השתתפו בחישוב $y_{r,s}$, וזאת בעזרת שידור הסקלר

$$\frac{\partial C}{\partial y_{r,s}} \text{ לממדיו של גרעין הקונבולוציה.}$$

```
def conv_backward(dCdY, K):
    p, q = K.size()
    Hy, Wy = dCdY.size()
    H = Hy + p - 1
    W = Wy + q - 1
    dCdX = torch.zeros((H, W))
    for r in range(Hy):
        for s in range(Wy):
            #forward: Y[r,s] = (X[r:r+p, s:s+q]*K).sum()
            #backward:
            dCdX[r:r+p, s:s+q] += dCdY[r, s]*K
    return dCdX
```

כעת נשכח את השימוש המקורי של פעולה זו ונעבור להשתמש בה כשכבה ברשת נוירונים. אם כן, נגדיר את הגרעין K כפרמטר נלמד של השכבה ונפעילה על batch של תמונות בהתפשטות לפנים ברשת. את ההתפשטות לאחור נשאיר למערכת הגזירה האוטומטית.

שכבות אלו מומשו ב־PyTorch במחלקה `nn.ConvTranspose2d` אשר מכלילה את הפעולה הנ"ל לתמונות בעלות כמה ערוצי קלט ופלט, באופן אנלוגי לשכבות קונבולוציה: לכל ערוץ פלט יהיה גרעין תלת־ממדי משלו, $C_{in} \times p \times q$, כאשר C_{in} הוא מספר ערוצי הקלט. הפעולה המתוארת בקטע הקוד הקודם תבוצע בנפרד עבור כל ערוץ קלט, ולבסוף כל התוצאות ייסכמו ליצירת ערוץ פלט יחיד. בהנחה שתמונת הקלט היא מממד $C_{in} \times H_y \times W_y$, חישוב הקונבולוציה המשוכללת עבור אחד מערוצי הפלט יתבצע כך:

```
output = torch.zeros((H, W))
for c in range(C_in):
    output += conv_backward(input[c, ...], K[c, ...])
```

ריבוי ערוצי הפלט יתקבל באמצעות חזרה על פעולה זו באופן בלתי תלוי עבור כל אחד מהם.

בשכבות המובנות של PyTorch מומשה גם הפונקציונליות של גודל הפסיעה והריפוד. אך יש לשים לב לכך שהפרמטר `stride` מתייחס לפעולת הקונבולוציה אשר ממנה נגזרת הקונבולוציה המשוכללת, עם התוצאה שממד האורך והרוחב של תמונת הפלט גדלים יחד עם ערך ה־`stride`, בניגוד להשפעת פרמטר זה על הקונבולוציה הרגילה.

עתה, בעזרת שכבות אלו נוכל לחבר ישירות את פלט המקודד הקונבולוציוני אל מפרש המורכב משכבות קונבולוציה משוכללת ולקבל רשת קונבולוציונית מלאה (Fully Convolutional Network – FCN). רשתות מסוג זה משמשות במגוון מטרות ראייה ממוחשבות מתקדמות, בין השאר כאשר ברצוננו לסווג את האובייקטים המופיעים בתמונת קלט, ונוסף על כך לזהות היכן הם נמצאים בתמונה.

שאלה לתרגול

אמנו מקודד עצמי מנקה רעש על אוסף הנתונים Fashion-MNIST המשתמש בשכבות קונבולוציה, קונבולוציה משוכללת ואקטיבציות לא לינאריות בלבד.

תרגום מכונה באמצעות רשת מקודד־מפרש נשנית

בפרק זה נלמד כיצד לשלב בארכיטקטורת מקודד־מפרש רכיבים של רשתות נשנות, ולאמץ לבצע תרגום אוטומטי של קטע טקסט נתון. ראשית נדון באוסף הנתונים המתאים למשימה זו: Tatoeba. זהו אוסף של זוגות משפטים בשפות שונות, שלהם משמעות זהה. המשפטים מתורגמים בהתנדבות בידי משתמשי הפרויקט, כך שמצד אחד אוסף הנתונים ממשיך להתעדכן כל הזמן, אך מצד אחר משתמשי האתר לרוב אינם מתרגמים במקצועם ולכן יש בו לא מעט שגיאות, קטנות וגדולות. לצרכינו ביחידה הנוכחית אוסף זה מספק, אך יש לזכור את מגבלותיו בבואנו לאמן רשת למטרות החורגות מהדגמת יכולת.

אחד היתרונות של אוסף זה הוא מגוון השפות הגדול הזמין בו. נשתמש שוב בספרייה Datasets לצורך טעינת הנתונים, כאשר לצורך הדוגמה שפת המקור שנבחר היא אנגלית, והיעד – צרפתית.

```
import datasets as ds
src="en"
tgt="fr"
dataset = ds.load_dataset("tatoeba", lang1=src, lang2=tgt)
```

הפרמטרים lang1 ו־lang2 שולטים בשפות של אוסף הנתונים המתקבל, ובאמצעות החלפתם תוכלו להשתמש בקוד המופיע בהמשך פרק זה גם לצורך אימון רשת לתרגום מעברית לאנגלית, למשל.

נתבונן בדגימה אחת מאוסף הנתונים המתקבל.

```
dataset["train"]["translation"][10]
```

פלט:

```
{'en': "Today is June 18th and it is Muiriel's birthday!",
 'fr': "Aujourd'hui nous sommes le 18 juin et c'est
l'anniversaire de Muiriel !"}

```

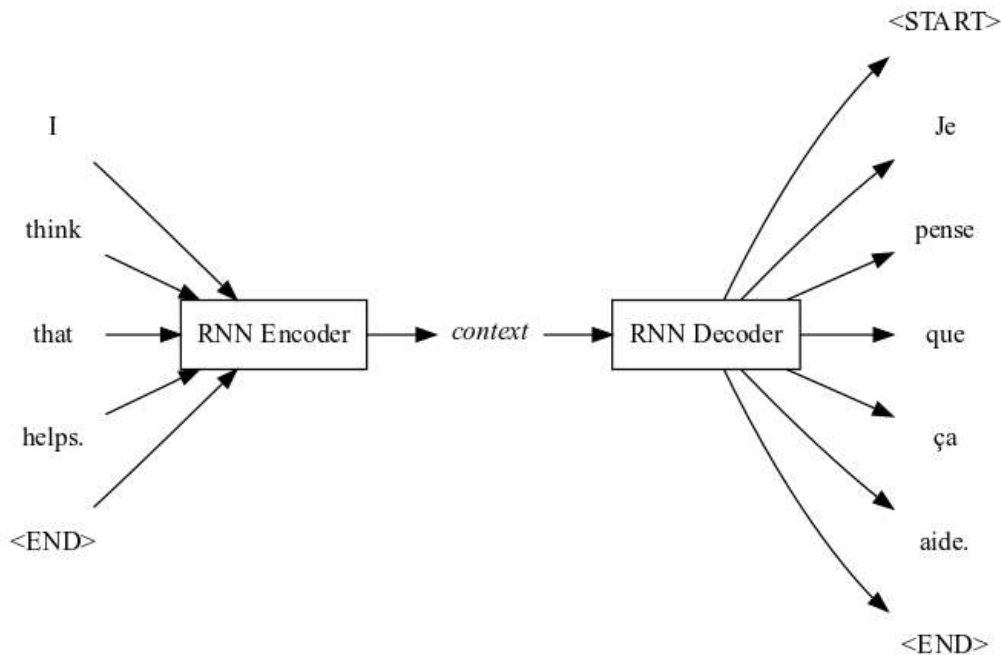
כרגיל במשימות עיבוד שפה טבעית, עלינו לבצע עיבוד מקדים לאוסף הנתונים לפני הזנתו לרשת נוירונים. מכיוון שכעת אנו עוסקים בשתי שפות, יש ליצור אוצר מילים שונה עבור כל שפה: אחד מהם ייבנה מכל המשפטים בשפת המקור, אנגלית, והשני – מכל המשפטים בשפת היעד, צרפתית. כמו כן, כפי שנראה מייד, ארכיטקטורת הרשת הנבחרת דורשת לאותת למקודד על סוף משפט קלט, והמפרש מצפה לקבל איתות נוסף גם על תחילתו של המשפט. על כן נוסיף לכל אוצר מילים טוקנים מתאימים. דוגמה למשפטים לאחר טוקניזציה מופיעה להלן.

```
Source:
['I', 'think', 'that', 'helps.', '<END>']
tensor([ 2, 140, 43, 4795, 1])
Target:
['<START>', 'Je', 'pense', 'que', 'ça', 'aide.', '<END>']
tensor([ 2, 4, 184, 39, 75, 644, 1])

```

הקלט לרשת יהיה הטנזור המספרי הראשון, והטנזור השני הוא הפלט הצפוי מהרשת, שבעזרתו נחשב את פונקציית המחיר.

באיור שלהלן מוצגת ארכיטקטורת הרשת ברמת ההפשטה הגבוהה ביותר: מקודד-מפרש אשר שני חלקיו מבוססים על רשתות נשנות.



המקודד בארכיטקטורה זו יקבל משפט מקור יחיד וימיר אותו לייצוג חבוי, הקרוי בהקשר הנוכחי וקטור ה"הקשר" (context). בידינו כבר כל הרכיבים לכתובת המקודד, זהו רכיב חילוץ המאפיינים ב-RNN אשר שימשה אותנו לסיווג הרגש המובע במשפט. השוו את קטע הקוד שלהלן לזה של הרשת FasterDeepRNNClassifier אשר כתבנו ביחידה 6, וודאו כי אתם מוצאים ביניהם דמיון.

```

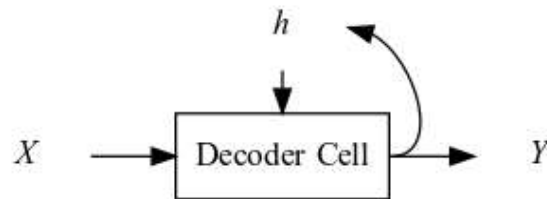
class Encoder(nn.Module):
    def __init__(self, embed_dim, hidden_dim, RNNlayers):
        super().__init__()
        self.src_embedding = nn.Embedding(len(src_vocab),
                                           embed_dim)

        self.rnn_stack = nn.LSTM(embed_dim,
                                  hidden_dim,
                                  RNNlayers)

    def forward(self, src_tokens):
        all_embeddings = self.src_embedding(src_tokens)
        all_embeddings = all_embeddings.unsqueeze(1)
        hidden_state_history, _ = self.rnn_stack(all_embeddings)
        context = hidden_state_history[-1, 0, :]
        return context
  
```

זכרו שפלט המודול LSTM הוא סדרת המצבים החבויים של התא הנשנה בראש ערימת ה-RNN, ולכן עבור וקטור ההקשר אנו בוחרים את האחרון שבה.

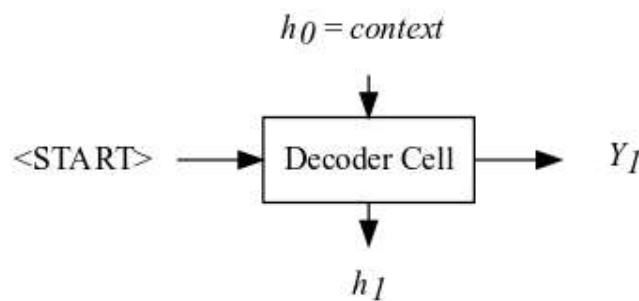
תכנון המפרש מציב לפנינו אתגר: עליו לייצר משפט בשפת היעד כפלט. נממש זאת בעזרת תא נשנה אשר יקבל כקלט את וקטור ההקשר מהמקודד ויבנה את משפט הפלט, טוקן אחר טוקן, מתחילת המשפט ועד סופו. על כן עלינו להוסיף לארכיטקטורת התא הנשנה הפשוט אפשרות לייצר פלט, כמאויר להלן.



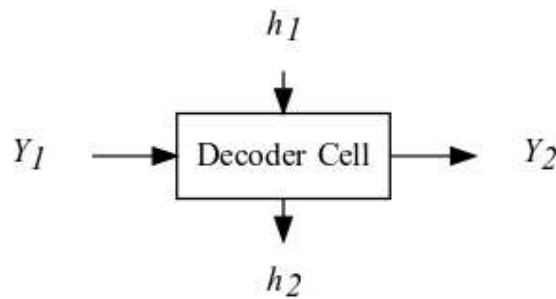
קלט התא X הוא טוקן (משוכן) בשפת היעד צרפתית, וכך גם פלט התא Y .

לפני כתיבת תא זה כמודול של PyTorch, נתאר את הדרך שבה ישתמש בו המפרש. ראשית, נדון במצב אימון שבו המפרש יקבל כקלט את וקטור ההקשר מהמקודד, וכן את סדרת הטוקנים של משפט המטרה, שנשמם $(Z_0, \dots, Z_T)$. זהו בן הזוג של משפט המקור שהוזן למקודד. נשים לב לכך שלפי העיבוד המקדים אשר את תוצאתו ראינו לעיל, Z_0 הוא תמיד הטוקן המייצג את תחילת המשפט, ו- Z_T מייצג את סוף המשפט. פלט המפרש יהיה גם הוא סדרת טוקנים, שנשמם $(Y_0, \dots, Y_T)$.

בתחילת החישוב הרקורסיבי בתא הנשנה נאתחל את המצב החבוי **בערכו של וקטור ההקשר** ונגדיר ידנית את Y_0 להיות טוקן תחילת המשפט. כעת נוזין טוקן זה בתור הקלט המסומן ב- X לעיל (לאחר שיכון כמובן), וכך נסמן למפרש שעליו להתחיל לתרגם את משפט המקור על סמך וקטור ההקשר. ראו את הצעד הראשון במפרש באיור המצורף.



כעת, לאחר קבלת הפלט Y_1 , נוזין אותו חזרה לתא, הפעם בתפקיד הקלט, ונקבל את Y_2 .



נחזור על פעולה זו עד לקבלת הטוקן Y_T , אשר כדי לייצרו נדרש לעדכן את המצב החבוי $T-1$ פעמים, ולייצר את כל הטוקנים הקודמים לו. ראו מימוש חישוב זה בקוד שלהלן.

```
class TrainingDecoder(nn.Module):
    def __init__(self, embed_dim, hidden_dim):
        super().__init__()
        self.tgt_embedding = nn.Embedding(len(tgt_vocab),
                                           embed_dim)
        self.RNNcell = DecoderRNNCell(embed_dim,
                                       hidden_dim)

    def forward(self, context, tgt_tokens):
        self.RNNcell.hidden_state = context
        translated_tokens = [START_Token]
        for idx in range(len(tgt_tokens)-1):
            previous_token = translated_tokens[idx]
            embedded_token = self.tgt_embedding(previous_token)
            predicted_token = self.RNNcell(embedded_token)
            translated_tokens.append(predicted_token.detach())
        return translated_tokens
```

שימו לב שאנו מאתחלים אובייקט `DecoderRNNCell` בתוך בנאי המפרש אולם נגדיר מחלקה זו רק בהמשך הפרק. לעת עתה ניתן להבין מהו הפלט הדרוש ממנו לפי השימוש בו במתודה `forward` והאיורים שלעיל.

ייתכן שבאחד משלבי הביניים הטוקן החזוי יהיה טוקן סיום המשפט, שהרי גם הוא חלק מאוצר המילים של שפת היעד. במקרה זה נפרש זאת כאיתות של הרשת על כך שהיא סיימה את עבודת התרגום, ועל כן נסיף במתודה `forward` אפשרות לסיים את החיזוי מוקדם מהצפוי, בעזרת שורות הקוד האלה.

```
if predicted_token == END_Token:
    break
```

לבסוף, נזכור שעל הרשת לעבור אימון, ולמטרה זו יש לבחור פונקציית מחיר מתאימה. מובן שנרצה לתגמל במחיר נמוך מודל אשר חזה נכונה טוקן Y_i הזהה לטוקן המתאים ממשפט המטרה Z_i , אך מעבר לכך, נרצה לתגמל במחיר נמוך עוד יותר מודל העושה זאת בביטחון רב. באופן דומה, נרצה לקנוס

מודל אשר כלל לא היה קרוב לחזות את Z_t במחיר גבוה יותר מאשר מודל אשר כמעט עשה זאת, אך בסוף "התבלבל".

שיקולים אלו מובילים אותנו לחשוב על בעיית יצירת הטוקן Y_t כבעיית סיווג קלאסית: התא הנשנה מקבל כקלט את המצב החבוי והטוקן הקודם, ועליו לייצר לכל טוקן באוצר המילים של שפת היעד את ההסתברות $P(Y_t = token)$. זהו למעשה פלט הפונקציה softmax המוכרת לנו. לבסוף, נבחר בתור הטוקן הבא במשפט המתורגם את זה בעל הסתברות הסיווג הגבוהה ביותר, ה־argmax. היתרון הגדול בנקודת מבט זו הוא שפונקציית המחר המבטאת את כל רצונותינו ידועה כבר, הלא היא האנטרופיה הצולבת.

כעת אנו מוכנים לפרט את התהליך החישובי המבוצע בתא הנשנה, המורכב משני שלבים:

1. חישוב המצב החבוי החדש. שלב זה יבוצע כבעבר, לפי הנוסחה

$$h_{t+1} = \tanh(W_{input}X + b_{input} + W_{hidden}h_t + b_{hidden})$$

כאשר X הוא הטוקן המתקבל כקלט, ו־ h_t הוא המצב החבוי הקודם.

2. חישוב הסתברויות הסיווג $P(Y = token)$ באמצעות הזנת המצב החבוי החדש לשכבה לינארית ומיידי אחריה – פונקציית softmax. אם כן, פלט התא יהיה וקטור בעל ערך לכל טוקן באוצר המילים, המחושב כך: $\text{softmax}(W_{out}h_{t+1} + b_{out})$.

הפרמטרים של התא הנשנה הם משקלי השכבות הלינאריות המופיעות לעיל וערכי ה־bias שלהן.

מימוש התא מופיע בקטע הקוד שלהלן.

```
class DecoderRNNCell(nn.Module):
    def __init__(self, embed_dim, hidden_dim):
        super().__init__()
        self.hidden_state = torch.zeros(hidden_dim)
        self.RNNcell = nn.RNNCell(embed_dim, hidden_dim)
        self.output_linear = nn.Linear(in_features=hidden_dim,
                                         out_features=len(tgt_vocab))
        self.logsoftmax = nn.LogSoftmax(dim=0)

    def forward(self, one_embedded_token):
        new_state = self.RNNcell(one_embedded_token,
                                   self.hidden_state)
        tgt_token_scores = self.output_linear(new_state)
        tgt_token_logprobs = self.logsoftmax(tgt_token_scores)

        self.hidden_state = new_state
        return tgt_token_logprobs
```

ראו כי השתמשנו בתא הנשנה המובנה ב־PyTorch, nn.RNNCell, אשר מבצע את החישוב הרשום בסעיף 1 לעיל.

עבודתנו עוד לא הסתיימה, שכן עלינו לחשב את המחיר עבור כל דגימה לאחר הזנתה ברשת. יהיה נוח לעשות זאת **תוך כדי** ההתפשטות לפנים ברשת, שכן כך לא נדרש לשמור את ההיסטוריה של וקטורי הסתברויות הסיווג של כל הטוקנים ($Y_0, \dots, Y_T$) עד לסוף תרגום המשפט. בעודנו זוכרים שהאנטרופיה הצולבת היא מינוס לוגריתם ההסתברות שהמפרש מקנה לטוקן הנכון, וכן שהטוקנים הנכונים נתונים במשתנה `tgt_tokens`, נוסיף גם את חישוב זה למתודה `forward` של `TrainingDecoder`, המופיעה בשלמותה להלן. תוספת חישוב המחיר מסומנת ב-# בקוד.

```
def forward(self, context, tgt_tokens):
    self.RNNcell.hidden_state = context
    translated_tokens = [START_Token]
    sentence_loss = 0
    for idx in range(len(tgt_tokens)-1):
        previous_token = translated_tokens[idx]
        embedded_token = self.tgt_embedding(previous_token)
        logprobs = self.RNNcell(embedded_token)
        predicted_token = logprobs.argmax()
        translated_tokens.append(predicted_token.detach())

        correct_token = tgt_tokens[idx+1] #
        token_loss = -logprobs[correct_token] #
        sentence_loss += token_loss #

    if predicted_token == END_Token:
        break
    return translated_tokens, sentence_loss
```

ראו כי כעת המפרש מחזיר למשתמש גם את המחיר הכולל של המשפט, שהוא סכום המחירים של כל אחד מהטוקנים במשפט המתורגם. מחיר זה יעניש מודל אשר חוזה טוקנים שגויים וכן מודל אשר מסיים עבודתו מוקדם מהצפוי, שכן אז הוא יחזה את טוקן סיום המשפט במקום הלא הנכון, וישלם על כך מחיר באנטרופיה הצולבת של טוקן זה.

כל שנותר לעשות כעת הוא לחבר את פלט המקודד אל המפרש, ולאמנם יחדיו. לאחר האימון, נרצה להשתמש ברשת לתרגום משפט בשפת המקור שעבורו אין לנו תרגום מוכן בסט האימון. על כן יש להוסיף למפרש מצב חיזוי, שבו הוא מקבל כקלט רק את וקטור ההקשר ומייצר טוקנים עד ליצירת טוקן סוף המשפט (או מספר מרבי של טוקנים קבוע מראש). מובן שבמצב חיזוי המפרש לא יחזיר את מחיר המשפט, שכן לא ניתן לחשבו כלל ללא משפט המטרה.

שאלות לתרגול

1. השלימו את שלב עיבוד הנתונים המקדים עבור שתי השפות, כך שלבסוף יתקבלו אוצר המילים של כל שפה וטנזורים המכילים את המספר הסיידורי של הטוקנים באוצר המילים, לכל משפט בכל שפה.
2. מהם ממדי הפרמטרים W_{out}, b_{out} המשמשים לחישוב טוקן הפלט בתא הנשנה החדש שהגדרנו לעיל?
3. כתבו בקוד את מצב החיזוי של המפרש, כמתואר בפסקה האחרונה בפרק זה.
4. א. חברו את המקודד והמפרש למודול PyTorch יחיד.

- ב. בחרו זוג שפות המוכר לכם וטענו את האוסף המתאים להן לזכרון המחשב.
- ג. אמנו רשת תרגום המכונה על batch קטן של זוגות משפטים.
- ד. הדפיסו כמה משפטים מתורגמים לצד התרגום הנכון שלהם, כפי שהוא מופיע באוסף הנתונים.
5. ניתן לאמן את המפרש באמצעות הזנת **הטוקן הנכון** לתא הנשנה בכל איטרציה במקום הטוקן האחרון שהמפרש חזה, וזאת כדי לייצר את הטוקן הבא במשפט המתורגם. מובן שאפשרות זו, הנקראת באנגלית teacher forcing, אפשרית רק במצב אימון, שכן רק אז בידינו הטוקנים הנכונים.
- א. הוסיפו את השימוש ב־teacher forcing למפרש, ואמנו שוב את הרשת על batch קטן.
- ב. ציירו על גרף אחד את המחיר הממוצע כפונקציה של epoch האימון עבור שני המודלים שאימנתם: בשאלה זו ובשאלה הקודמת.

שכבת תשומת לב ושימוש במפרש הנשנה

אחד האתגרים המרכזיים בשימוש ברשתות נוירונים נשנות הוא למידת קשרים ארוכי טווח. דנו בבעיה זו כבר ביחידה 6. בבואנו לבצע תרגום מכונה, בעיה זו מקבלת משנה תוקף. חשבו לדוגמה על זוג המשפטים:

"I have a dog, and I love it"
 "יש לי כלב, ואני אוהבת אותו"

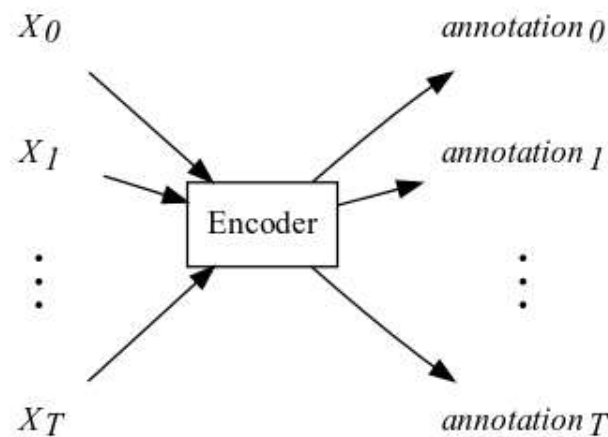
כדי לתרגם נכונה מאנגלית לעברית את המילה האחרונה בזוג זה, יהיה על הרשת לזהות שהמילה "אותו" מתייחסת ל-"dog", ולדעת שמילה זו בעברית היא ממין זכר. ברשת שבנינו בפרק הקודם, קשר זה בהכרח חייב להישמר דרך כל החישובים המבוצעים במקודד לאחר הזנת המילה "dog", וכן דרך כל החישובים המבוצעים במפרש עד להזנת המילה "אותו", שכן המקודד מעבד את משפט המקור מתחילתו לסופו, מייצר את וקטור ההקשר, ולבסוף המפרש בונה את המשפט המתורגם, שוב מההתחלה אל הסוף.

כדי להקל על הרשת בלמידת קשרים מסוג זה, נרצה לספק למפרש גישה מיידית לכל המילים במשפט המקור, כך שבדוגמת התרגום הקודמת היא תוכל (באופן אידיאלי) להבין בעת עבודתה שהמילה האחרונה במשפט צריכה להיות כינוי גוף; וכדי להחליט אם מדובר ב-"הוא", "היא" או "זה", היא תוכל לחפש את שם העצם המתאים במשפט המקור ישירות, מבלי להסתמך על וקטור ההקשר שהוזן לתא הנשנה באיטרציה הראשונה שלו.

פונקציונליות חיפוש זו, הקרויה שכבת תשומת לב (attention layer), היא הבסיס לארכיטקטורת רשת מוצלחת במיוחד, ה-"transformer", ונמצאת בשימוש במשימות רבות מעבר לתרגום מכונה, ואף מחוץ לתחום של עיבוד שפה טבעית. בפרק זה נכיר צורה פשוטה שלה, ונראה כיצד לשלבה בין המפרש למקודד ברשת תרגום המכונה במטרה להקל על הרשת בלמידת הקשרים של מילה במשפט היעד למילים במשפט המקור.

המקודד

ראשית, עלינו לשנות את פלט המקודד שכתבנו בפרק הקודם: ברצוננו לייצר כעת ייצוג חבוי של כל **טוקן** במשפט המקור, כך שבמקום וקטור הקשר יחיד המסכם את המשפט כולו, המקודד יעביר כפלט וקטור אנוטציה (annotation) **עבור כל אחד** מהטוקנים במשפט המקורי. לאחר אימון מוצלח, וקטורים אלו יהיו **סיכום של הטוקן בהקשרו** בתוך המשפט הנתון. נאייר זאת להלן.



למזלנו, כדי לממש פונקציונליות זו בקוד יש לשנות רק שורה אחת במקודד מהפרק הקודם. זכרו שפלט המודול `nn.LSTM` הוא היסטוריית המצבים החבויים של השכבה העליונה בערימת התאים. עד כה העברנו הלאה רק את המצב החבוי האחרון, השייך לטוקן הקלט האחרון, אך כעת נשתמש בכל ההיסטוריה: המצב החבוי h_t יהיה האנוטציה של הטוקן X_t . ראו את המקודד החדש בקטע הקוד שלהלן, כאשר השורות הנבדלות מהמקודד הקודם מסומנות ב-#.

```
class Encoder(nn.Module):
    def __init__(self, embed_dim, hidden_dim, RNNlayers):
        super().__init__()
        self.src_embedding = nn.Embedding(len(src_vocab),
                                           embed_dim)

        self.rnn_stack = nn.LSTM(embed_dim,
                                  hidden_dim,
                                  RNNlayers)

    def forward(self, src_tokens):
        all_embeddings = self.src_embedding(src_tokens)
        all_embeddings = all_embeddings.unsqueeze(1)
        annotations, _ = self.rnn_stack(all_embeddings)
        return annotations.squeeze()
```

המפרש

שוב נשנה את הקוד שכתבנו בפרק הקודם, וכעת נאפשר למפרש גישה ישירה אל האנוטציות: לפני כל צעד עדכון של התא הנשנה נחשב **וקטור הקשר פרטי** בעזרת שכבת תשומת הלב, שנלמד עליה בפירוט בהמשך הפרק. לעת עתה נסתפק בתובנה ששכבה זו תקבל כקלט את המצב החבוי הנוכחי של המפרש, ותחשב עבורו ממוצע משוקלל של האנוטציות, כאשר טוקני משפט המקור הרלוונטיים ביותר לצעד המפרש הנוכחי יזכו לבכורה בממוצע זה. בנוסחה,

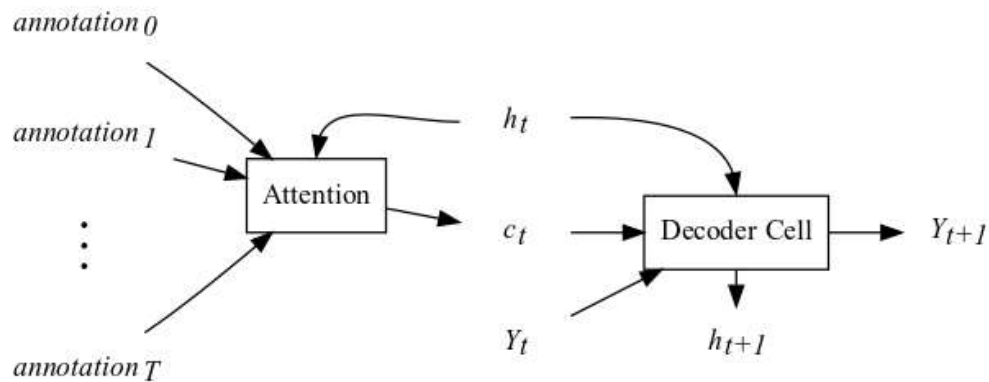
$$c_t = \sum_{k=0}^T \alpha(h_t, \text{annotation}_k) \cdot \text{annotation}_k$$

α היא הפונקציה שלפיה מתקבלת ההחלטה אם האנוטציה ה- k ית רלוונטית ליצירת הטוקן הבא בתור במשפט הפלט. אם זהו המצב, על משקלה של אנוטציה זו בווקטור ההקשר הנוכחי להיות גבוה. שימו

לב שלפי נוסחה זו ממד c_t זהה לזה של האנוטציות, שממדן כממד המרחב החבוי של המקודד. לאחר קבלת וקטור ההקשר הפרטי, התא הנשנה ישלבו בצעד עדכון המצב החבוי כדלהלן,

$$h_{t+1} = \tanh(W_{input}X + b_{input} + W_{hidden}h_t + b_{hidden} + W_c c_t + b_c)$$

מלבד תוספת זו, החישוב בתא הנשנה בעל מנגנון תשומת הלב נשאר זהה. נסיים דיון זה באיור סכמתי של התהליך החישובי של איטרציה יחידה במפרש, ומימוש התא החדש בקוד.



```
class AttnDecoderRNNCell(nn.Module):
    def __init__(self, embed_dim, hidden_dim):
        super().__init__()
        self.hidden_state = torch.zeros(hidden_dim)
        self.RNNcell = ContextRNNCell(embed_dim, hidden_dim)
        self.output_linear = nn.Linear(in_features=hidden_dim,
                                         out_features=len(tgt_vocab))
        self.logsoftmax = nn.LogSoftmax(dim=0)

    def forward(self, one_embedded_token, attn_context):
        new_state = self.RNNcell(one_embedded_token,
                                   self.hidden_state,
                                   attn_context)
        tgt_token_scores = self.output_linear(new_state)
        tgt_token_logprobs = self.logsoftmax(tgt_token_scores)
        self.hidden_state = new_state
        return tgt_token_logprobs
```

השוו תא זה לתא הנשנה DecoderRNNCell מהפרק הקודם וראו כי החידוש בא לידי ביטוי בשורות המסומנות ב-# בלבד. בראשונה שבהן אנו מאתחלים תא אלמן מסוג ContextRNNCell, אשר מבצע את עדכון המצב החבוי על סמך שלושה וקטורי קלט, החדש שבהם הוא ההקשר הפרטי, c_t . ניתן לראות זאת גם בשורה השנייה המסומנת, שם התא מקבל קלט נוסף: פלט שכבת תשומת הלב.

כדי לשלב תא זה בתוך המקודד ולייצר את טוקני המשפט המתורגם, זה אחר זה, יהיה עלינו לחשב ולהעביר לו את וקטור ההקשר בכל איטרציה, כפי שניכר מהפרמטרים של המתודה forward שלו. נעשה זאת בעזרת שכבת תשומת הלב, וכעת נדון בה בפירוט.

תשומת לב

מנגנון תשומת הלב שואב השראה מחיפוש במאגר נתונים, ולפיכך נפתח את הדיון בדוגמה זו. הניחו כי ברשותנו מילון המורכב מזוגות של מפתחות וערכים, למשל,

```
values = ["woman", "man", "bicycle", "queen", "house"]
keys    = values
my_dict = dict(zip(keys, values))
pprint(my_dict)
```

פלט:

```
{'bicycle': 'bicycle',
 'house': 'house',
 'man': 'man',
 'queen': 'queen',
 'woman': 'woman'}
```

ראו כי במילון זה השתמשנו בערכים עצמם בתור המפתחות לצורך פשטות הדוגמה.

כשנרצה לחפש מילה במילון זה נבצע השוואה של שאילתת החיפוש למפתחות (למעשה ההשוואה מתבצעת לאחר מעבר בפונקציית גיבוב, איך אין זה רלוונטי עבורנו), ואם קיים ערך במילון בעל מפתח זה בדיוק, הוא יוחזר. אם אין במילון מפתח זהה לשאילתה, לא יוחזר דבר. ראו דוגמה לכך להלן.

```
print(my_dict.get("woman"), my_dict.get("girl"))
```

פלט:

```
woman None
```

למטרתנו, נרצה שהחיפוש המילוני יהיה "רך": למשל ביודענו שהמילה "girl" בעלת קשר סמנטי הדוק למילה "woman", נצפה לראותה כתוצאת החיפוש, אף שמפתח זהה בדיוק לשאילתה "girl" לא קיים במילון. בבואנו לממש רצון זה עומדים לפנינו שני אתגרים:

1. המרת המפתחות והשאילתות לייצוג המביא לידי ביטוי את הקשר הסמנטי בין מילים.
2. מדידת הדמיון בין שתי מילים על סמך ייצוג זה, שכן נרצה להחזיר כפלט החיפוש את המילה מהמילון שדומה ביותר לשאילתה.

המשימה הראשונה נראית מאתגרת על פניה, אך לאחר מחשבה נוכל להסיק שכבר ראינו את פתרונה – אלו הם שיכוני המילים המאומנים מראש של GloVe, שפגשנו לראשונה כאשר עסקנו בעיבוד מקדים של נתוני טקסט ביחידה 6. נייבא אותם שוב, וכעת נגדיר את מפתחות המילון שלנו להיות וקטורי השיכון של המילים המתאימות.


```
from torchtext.vocab import GloVe
glove_embedder = GloVe(name='6B', dim=50)
keys = glove_embedder.get_vecs_by_tokens(values)
print(keys.size(), keys.dtype)

torch.Size([5, 50]) torch.float32
```

פלט:

שימו לב שזאת מפתחות המילים במילון הם וקטורים ממשיים במרחב בעל 50 ממדים.

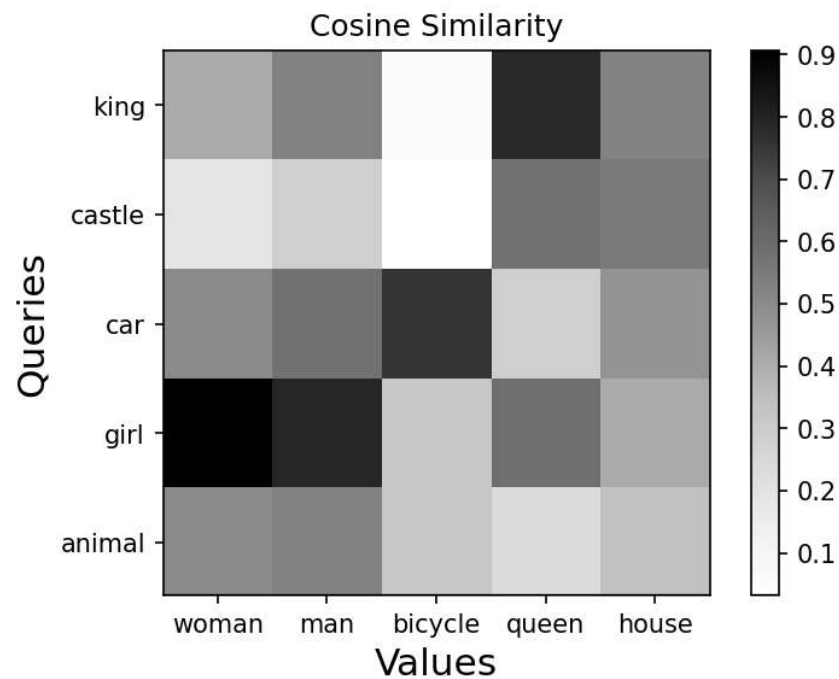
כאשר בידינו שיכון המילים, פתרון הבעיה השנייה נעשה פשוט – כל מדד של דמיון או קרבה בין שני וקטורים יבצע עבודה סבירה. כנהוג בשימושי עיבוד שפה טבעית, נבחר לעת עתה את דמיון הקוסינוס (Cosine similarity) כמדד לדמיון הסמנטי. על בסיס השיכונים של שתי מילים, שניהם וקטורים של מספרים ממשיים באותו מרחב, פונקציית דמיון זו מחשבת את **קוסינוס הזווית** בין שני הווקטורים, לפי הנוסחה הזו:

$$S_c(q, k) = \frac{q \cdot k}{\|q\| \|k\|}$$

הפונקציה מחשבת את המכפלה הפנימית של וקטורי הקלט ומנרמלת אותם כך שיתקבל פלט בטווח $[-1, 1]$. ככל שהערך $S_c(q, k)$ גדול יותר, כך הזווית בין הווקטורים קרובה יותר לאפס, ולפיכך השאילתה q דומה יותר למפתח k . בעזרת פונקציה זו נממש את החיפוש הרך במילון בקוד.

```
def soft_search(query, keys, values):
    dict_size = len(values)
    cos_similarity = torch.zeros(dict_size)
    q_norm = query.norm()
    for idx in range(dict_size):
        dot = (query*keys[idx]).sum()
        k_norm = keys[idx].norm()
        cos_similarity[idx] = dot/(q_norm*k_norm)
    best_match_idx = cos_similarity.argmax()
    return values[best_match_idx], cos_similarity
```

ראו כי פונקציה זו מצפה לקבל את השאילתה המשוכנת באמצעות GloVe בפרמטר `query` וסדרה של מפתחות בפרמטר `keys`; אלו יהיו שיכוני המילים הנמצאות בפרמטר `values`. תוצאות החישוב עבור שאילתות אחדות לדוגמה מאוירות להלן, כאשר הערך הכהה ביותר מצביע על תוצאת החיפוש עבור השאילתה בשורה זו.



ממבט באיור שלעיל, נסיק שלעתיים ישנן כמה תשובות הולמות עבור שאילתה נתונה, למשל ל-"king" קשר סמנטי חזק ל-"queen", כמובן, אך גם ל-"man". במימוש הנוכחי, תוצאת החיפוש אינה מביאה זאת בחשבון, שכן התוצאה הדומה ביותר בלבד היא שמוחזרת כפלט. נתקן זאת באמצעות חישוב ממוצע משוקלל עבור הפלט, ובהזדמנות זו גם נוותר על חישוב הנורמות, במטרה לייעל את זמן הריצה של הפונקציה. התוצאה המתקבלת היא פונקציית תשומת לב המכפלה הפנימית (dot product attention).

```
def dot_attention(q, K, V):
    dict_size = K.size(0)
    similarity = torch.zeros(dict_size)
    for idx in range(dict_size):
        similarity[idx] = (q*K[idx,:]).sum()
    attn_weights = F.softmax(similarity,dim=0)
    weighted_V = torch.zeros_like(V)
    for idx in range(dict_size):
        weighted_V[idx,:] = attn_weights[idx]*V[idx,:]
    output = weighted_V.sum(dim=0)
    return output, attn_weights
```

ננתח פעולת פונקציה זו בפרוטרוט:

- ראשית, כמו קודם, אנו מחשבים מדד דמיון בין השאילתה q לבין כל אחד מהמפתחות, שהם עמודותיה של המטריצה K .
- אחרי כן אנו מנרמלים את מדד הדמיון בעזרת הפונקציה softmax , שכן ברצוננו להשתמש בו כמשקל לחישוב ממוצע משוקלל.
- התוצאה המתקבלת ב- attn_weights היא וקטור שסכום איבריו הוא 1, וככל שהשאילתה דומה יותר למפתח מסוים, כך משקלו של וקטור זה עולה בתוצאת החיפוש.

- לבסוף, ניכר כי הפלט של פונקציית חיפוש זו הוא וקטור יחיד, הממוצע המשוקלל של עמודות המטריצה V . אלו הם הערכים במילון המתאימים למפתחות ב- K .

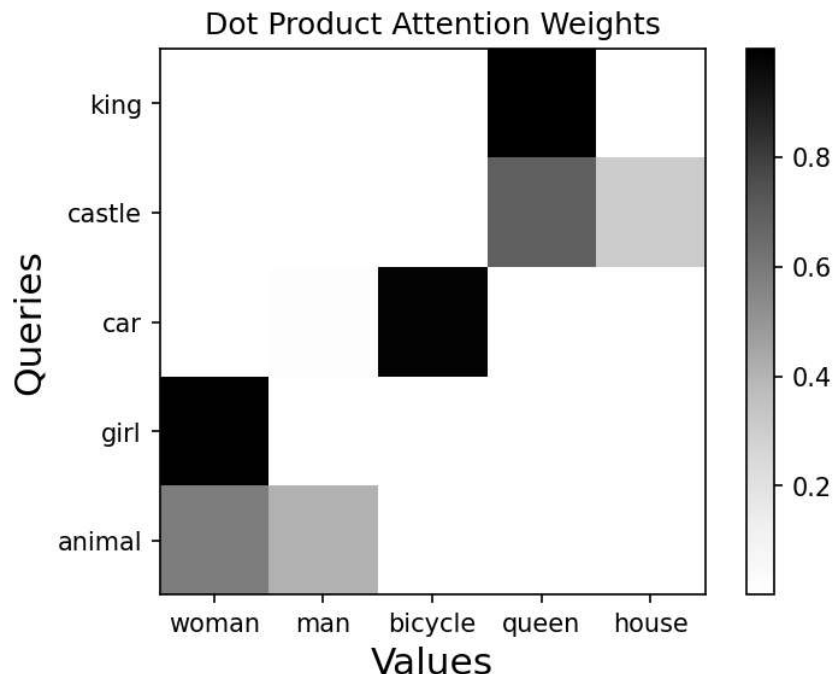
שימו לב שבעת גם על ערכי המילון להיות וקטורים ממשיים, כדי לאפשר את חישוב הממוצע. לפיכך, עבור דוגמת המילון שלנו, נעביר לפונקציית תשומת הלב את המילים לאחר השיכון באמצעות GloVe, כך שהמטריצות V ו- K יהיו זהות. בקטע הקוד שלהלן נדגים את השימוש בפונקציה.

```
src_words = ["woman", "man", "bicycle", "queen", "house"]
values = glove_embedder.get_vecs_by_tokens(src_words)
keys = values
query = glove_embedder.get_vecs_by_tokens("king")
search_result, attn_weights = dot_attention(query, keys, values)
print(search_result.size(), search_result.dtype)
print(attn_weights)
```

פלט:

```
torch.Size([50]) torch.float32
tensor([[7.0899e-05, 9.9381e-04, 1.3337e-09, 9.9831e-01, 6.2366e-04]])
```

ראו כי תוצאת החיפוש אף היא וקטור באותו מרחב שיכון בעל 50 ממדים. במבט למשקלי הממוצע ניכר כי למילה "queen" ההשפעה הרבה ביותר על הערך המתקבל, באופן המתאים לדמיון הסמנטי בין מילה זו לשאילתה. להלן נאייר כמה תוצאות נוספות של שימוש בפונקציית תשומת לב המכילה הפנימית.



מהאיור ניכר למשל שתוצאת החיפוש עבור השאילתה "castle" היא בעיקרה ממוצע של שיכוני המילים "queen" ו-"house", כצפוי.

חישוב תשומת הלב במפרש

כעת ברשותנו כל הדרוש כדי לתאר ולממש את החישוב המבוצע במפרש אשר ישמש לתרגום המשפט. נניח מאחורינו את דוגמת חיפוש המילים במילון וניזכר שעלינו לייצר עתה בכל איטרציה וקטור הקשר פרטי עבור התא הנשנה. וקטור זה יהיה תוצאת החיפוש של שכבת תשומת לב המכפלה הפנימית עם הקלט הזה: הערכים והמפתחות של השכבה יהיו וקטורי האנוטציה שיצר המקודד, והשאלתה תהיה המצב החבוי הנוכחי. אם כן, חישוב תשומת הלב המבוצע לפני יצירת הטוקן ה- t במשפט המתורגם הוא:

$$c_t = \sum_{k=0}^T \alpha(h_t, \text{annotation}_k) \cdot \text{annotation}_k$$

כאשר

$$\alpha(h_t, \text{annotation}_k) = \text{softmax} \begin{pmatrix} h_t \cdot \text{annotation}_0 \\ h_t \cdot \text{annotation}_1 \\ \vdots \\ h_t \cdot \text{annotation}_T \end{pmatrix}$$

שימו לב שכדי לבצע חישוב זה ללא תקלה, ממד המצב החבוי של המפרש, h_t , צריך להיות זהה לזה של המקודד, זהו ממד וקטורי האנוטציות. בשימוש בתשומת לב אנו מאפשרים לרשת לייצג את המצב החבוי h_t בצורה דומה לאנוטציות הרלוונטיות לתרגום הטוקן הבא בתור. כמובן, כדי לנצל אפשרות זו יהיה על הרשת ללמוד לעשות זאת בתהליך האימון. קוד המפרש מופיע בנספח בסוף הפרק. התבוננו בו וראו כי הוא דומה ברובו למפרש ללא תשומת הלב, כפי שכתבנו בפרק הקודם.

לבסוף נחבר את המפרש למקודד ונקבל את הרשת המלאה, כדלהלן.

```
class Translator(nn.Module):
    def __init__(self, embed_dim, hidden_dim, encoder_layers):
        super().__init__()
        self.encoder = Encoder(embed_dim,
                                hidden_dim,
                                encoder_layers)
        self.decoder = AttnDecoder(embed_dim, hidden_dim)
    def forward(self, src_tokens, tgt_tokens):
        annotations = self.encoder(src_tokens)
        return self.decoder(annotations, tgt_tokens)
```

נסיים פרק זה בהסתייגות: במימוש ארכיטקטורת הרשת שמנו דגש על בהירות המימוש ופשטות החישוב, וזאת על פני יעילותו ואפקטיביות המודל הנלמד. וביתר פירוט:

- השתמשנו באוסף נתונים אשר בנו מתנדבים, שידוע שיש בו טעויות.
- בעת העיבוד המקדים ביצענו טוקניזציה בצורה הפשוטה ביותר האפשרית.
- השתמשנו במפרש בעל שכבה יחידה של RNN המבוססת על תא אלמן, התא הנשנה הפשוט ביותר.
- מימשנו את החישוב הרקורסיבי בתא הנשנה של המפרש וכן את החישוב המבוצע בשכבת תשומת הלב באמצעות לולאות פייתון, אשר השימוש בהן אינו יעיל.
- השתמשנו בצורה הפשוטה ביותר של שכבת תשומת הלב, שלה גרסאות רבות ומתוחכמות יותר.

התוצאה המתקבלת מהחלטות אלו היא שהרשת שלנו אמנם מסוגלת ללמוד, אך היא עושה זאת באיטיות רבה ואין היא מספקת תוצאות טובות במיוחד. על כן, לאחר קריאת פרק זה וקודמו עליכם להבין את עקרון הפעולה של רכיביה, ועם זאת לזכור שדרושים שיפורים טכניים ותיאורטיים נוספים כדי למצות את מלוא יכולתם.

שאלות לתרגול

1. כתבו את המחלקה `ContextRNNCell` המממשת תא אלמן בעל שלושה וקטורי קלט שונים: המצב החבוי הקודם, טוקן הקלט הנוכחי ווקטור ההקשר הפרטי.
רמז: השתמשו בשכבות לינאריות לכל אחד מווקטורי הקלט, ולבסוף הפעילו עליהם את פונקציית האקטיבציה הדרושה. השקיעו מחשבה בממדי השכבות הלינאריות.
2. כתבו את פונקציית תשומת לב המכפלה הפנימית בצורה וקטורית, ללא שימוש בלולאות.
3. תכננו וכתבו את מצב החיזוי של המפרש עם תשומת הלב.
רמז: שאבו השראה ממצב החיזוי של המקודד מהפרק הקודם.
4. הניחו שנתון משפט קלט באורך T_1 טוקנים אשר עברו אובייקט `Translator` מייצר T_2 טוקנים כפלט. כמה פעמים המפרש קורא לפונקציית תשומת הלב? כמה פעמים מבוצע החישוב $q \cdot k$ עבור שאילתה ומפתח יחידים?

נספח: קוד המפרש בעל שכבת תשומת לב

```

class AttnDecoder(nn.Module):
    def __init__(self, embed_dim, hidden_dim):
        super().__init__()
        self.hidden_dim = hidden_dim
        self.tgt_embedding = nn.Embedding(len(tgt_vocab),
                                           embed_dim)

        self.RNNcell = AttnDecoderRNNCell(embed_dim,
                                           hidden_dim)

        self.attn_layer = dot_attention
    def forward(self, annotations, tgt_tokens):
        self.RNNcell.hidden_state = torch.zeros(self.hidden_dim)
        keys = annotations
        values = annotations
        translated_tokens = [START-Token]
        sentence_loss = 0
        for idx in range(len(tgt_tokens)-1):
            previous_token = translated_tokens[idx]
            embedded_token = self.tgt_embedding(previous_token)
            query = self.RNNcell.hidden_state
            attn_context, _ = self.attn_layer(query, keys, values)
            logprobs = self.RNNcell(embedded_token,
                                    attn_context)

            predicted_token = logprobs.argmax()
            translated_tokens.append(predicted_token.detach())

            correct_token = tgt_tokens[idx+1]
            token_loss = -logprobs[correct_token]
            sentence_loss += token_loss

        if predicted_token == END-Token:
            break
        return translated_tokens, sentence_loss

```



מהדורה פנימית
לא להפצה ולא למכירה
מק"ט 22961-0000